Strategy synthesis of LIA-definable impartial combinatorial games

Liangda Fang[a,d], Meihong Yang[a], Jiahao Luo[a], Shiguang Feng[b], Liping Xiong[c], Dingliang Cheng[a], Yunlai Hao[a], Quanlong Guan[a]

[a] *Jinan University, Guangzhou, 510632, Guangdong, China*
[b] *Sun Yat-sen University, Guangzhou, 510006, Guangdong, China*
[c] *Wuyi University, Jiangmen, 529020, Guangdong, China*
[d] *Pazhou Lab, Guangzhou, 510335, Guangdong, China*

Abstract

Strategy representation and reasoning has recently received much attention in artificial intelligence. Impartial combinatorial games (ICGs) are a type of elementary and fundamental games in game theory. One of the challenging problems of ICGs is to construct winning strategies, particularly, generalized winning strategies for possibly infinitely many instances of ICGs. In this paper, we investigate synthesizing generalized winning strategies for ICGs. To this end, we first propose a logical framework to formalize ICGs based on the linear integer arithmetic fragment of numeric part of PDDL. We then propose two approach to generating the candidate formula that exactly captures the states in which the player can force to win: one adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Furthermore, we compute winning strategies for ICGs based on the winning formula. Experimental results on several games demonstrate the effectiveness of our approach.

Keywords: Impartial Combinatorial Games, Strategy Synthesis, Partial MaxSAT, Linear Integer Arithmetic

1. Introduction

Strategy representation and reasoning has recently received much attention in artificial intelligence, particularly, multi-agent systems and game theory [**? ?**

?]. In the area of game theory, one class of the elementary and fundamental games is impartial combinatorial games (ICGs) that studies sequential games with perfect information [**?**]. Various ICGs (*e.g.*, Nim [**?**], Wythoff [**?**], Chomp [**?**] and Empty-and-Divide [**?**]), and their variants and extensions [**? ?**] have been proposed and analyzed. The following is Take-away-3 game [**?**].

Example 1. Initially, a pile of chips is on a table. In a move, a player can take at most 3 chips from the table and must take at least 1 chip. The loser is the player who has no chip can be taken. □

In general, an ICG contains infinite number of states. Infinite-state ICGs pose two challenging problems. The first problem is to identify the exact characteristic of the set of winning states, that are the states in which the player can force to win. This problem has always been a long-standing topic of ICGs [**? ? ? ?**]. For example, a player can force to win in any state where the number v of remaining chip is not divisible by 4 (more precisely, $v \not\equiv_4 0$) in Take-away-3 game. It is impossible that a player have a winning strategy in other states where $v \not\equiv_4 0$.

The second problem is to synthesize generalized winning strategies for infinite many winning states. An ICG instance is an ICG together with a specific initial state. A winning strategy for an ICG instance can be computed via backward search algorithm [**?**]. However, this strategy holds for only some instances but not (infinitely) many instances. This is illustrated in the Take-away-3 game as follows. Assume that the number of chips is initially 7. In the first round, the first player takes 3 chips. If the other player takes 1 (resp. 2 and 3) chips, then she chooses to take 3 (resp. 2 and 1) chips in the next round. Following the above strategy, the first player finally wins no matter how the other player does. But for another instance where the initial state is $v = 5$, after the first player takes 3 chips, the number of the remaining chips is 2. Then, the other player takes 2 chips and no chips is on the table. Finally, the loser is the first player. However, the first player can force to win in the initial state $v = 5$ since $5 \not\equiv_4 0$. Hence, the above strategy does not work for some of these instances.

In this paper, we propose an automatic approach to solving the above two problems of infinite-state ICGs. The contributions of this paper contain:

1. We propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [**?**].
2. We define an arithmetic formula, called the winning formula, that is a symbolic representation that exactly captures the states in which the player forces to win. We propose a counterexample-guided approach to synthesizing the winning formula. It first attempts to generate a candidate formula ϕ via a finite set of states, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the winning formula ϕ. Otherwise, a set of new states that are witnesses of its failure will be provided, and used to create a new candidate formula. The above steps repeat until the winning formula is discovered.
3. One crucial task of the counterexample-guided approach is to generate a candidate formula that is consistent with two generated sets of winning states and losing states. We design two methods to accomplish this task. The first method we adopt is a simple enumerative approach proposed in [**?**]. The main idea is to generate candidate formulas by induction on the size until it finds a desired one. However this approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula is relatively large. To accelerate the generation of candidate formulas, we propose the second method, and formulate it as an exact binary classification problem with all atoms as classification attributes and encode each classification problem as an instance of the partial MaxSAT problem. We can efficiently acquire the winning formula of large size with the aid of off-the-shelf partial MaxSAT solvers.
4. The another core task of the counterexample-guided approach is to check if the candidate formula is the winning formula. We design several con-

straints for the winning formula that is represented by linear integer arithmetic. The candidate formula is correct iff all of the constraints are valid. Therefore, this task can be accomplished by the state-of-the-art SMT solvers: Z3 [**?**], CVC4 [**?**] and MathSMT 5 [**?**].

5. To synthesize the winning strategy, we develop a counterexample-guided approach from a generated winning formula. It first produces a finite set of actions that transforms at least one winning state to a losing state, and then determines the condition formula that captures the set of winning states from which performing the action leads to a losing state for each action. A rule is a pair of an action and its corresponding condition formula, and a strategy consists of a set of rules. To check the correctness of a candidate strategy, we design the constraints for each of its rule.
6. We establish a standard benchmark of ICGs composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. Each test case is formalized in a PDDL file. We also evaluate our proposed approach on this benchmark, demonstrating that the effectiveness of our proposed approach.

This paper is an extended and revised version of [**?**] and [**?**]. We provide the complete proofs of all propositions, lemmas, and theorems in [**?**] together with Theorems 3 and 4. Theorem 3 establishes the undecidability of determining if an LIA-definable game is an LIA-definable ICG, whereas Theorem 4 shows the decidability of determining if a state is winning in an LIA-definable ICG. In [**?**], the strategy synthesis approach first splits it into a set of arithmetic terms such that their disjunction is equivalent to the winning formula according to a set of simple division rules. Then, for each term, it uses the enumerative algorithm to obtain the desired action such that the player reaches a losing state from any state satisfying the term via performing this action. However, it is not guaranteed that each term has a proper action, and the above approach fails to compute a winning strategy. In addition, the experimental results show that the strategy synthesis approach proposed in [**?**] combined with the par-

tial MaxSAT-based method to generate candidate formulas outperforms the one in [**?**]. Hence, this paper does not include the illustration and experimental comparison of the strategy synthesis approach proposed in [**?**].

(Structure ??) The rest of the paper is organized as follows. In Section 2, we provide some necessary preliminaries, including the concepts of linear integer arithmetic (LIA), the maximum satisfiability problem, and ICGs. In Section 3, we propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [**?**]. In Section 4, we introduce the exact binary classification problem and propose two approach that reduce this problem to the enumerative approach proposed in [**?**] and partial MaxSAT problems, respectively. In Section 5, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas and strategies for impartial combinatorial games. In Section 6, we evaluate our proposed approaches on the publicly available benchmark composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. (??) Finally, we offer related works in Section 7, and close with conclusions and future works in Section 8.

2. Preliminaries

In this section, we introduce the syntax of linear integer arithmetic (LIA), the partial maximum satisfiability (MaxSAT) problem, and impartial combinatorial games (ICGs).

2.1. Linear integer arithmetic

Let $\mathcal{N}$ be the set of integers and $\mathcal{V}$ the set of variables. *Linear integer arithmetic expressions* (`Exp`), *atoms* (`Atom`) and *formulas* (`Form`) are defined by the following grammar:

$$e, e_1, e_2 \in \texttt{Exp} :: c \mid v \mid e_1 + e_2 \mid e_1 - e_2$$

$$l \in \texttt{Atom} :: e_1 = e_2 \mid e_1 < e_2 \mid e_1 \leq e_2 \mid e \equiv_{c_1} c_2$$

$$\phi, \phi_1, \phi_2 \in \texttt{Form} :: \top \mid \bot \mid l \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \forall v\phi \mid \exists v\phi$$

where $c, c_1, c_2 \in \mathcal{N}$ and $v \in \mathcal{V}$.

The atom $e \equiv_{c_1} c_2$ means that e and c_2 are congruence modulo c_1, that is, $e - c_2$ is divisible by c_1. We use $e_1 \neq e_2$ for $\neg(e_1 = e_2)$, $e_1 > e_2$ for $\neg(e_1 \leq e_2)$, $e_1 \geq e_2$ for $\neg(e_1 < e_2)$, $e \not\equiv_{c_1} c_2$ for $\neg(e \equiv_{c_1} c_2)$, and $\phi_1 \rightarrow \phi_2$ for $\neg\phi_1 \vee \phi_2$. We use $|e|$ (resp. $|\phi|$) to represent the size of an LIA expression e (resp. LIA formula ϕ), that is, the number of occurrences of integers, variables and connectives in e (resp. ϕ). For example, the formula $x = 3$ has size 3. Given a set Ψ of atoms, we say a formula ϕ is based on Ψ, iff it is built from the set Ψ, the connectives $\neg, \vee, \wedge, \rightarrow$ and two Boolean constants $\top$ and $\bot$. For example, suppose that $\Psi = \{x = 1, x > 2\}$. The formula $\neg(x = 1 \vee x > 2)$ is based on Ψ, but $x \leq 3 \wedge x > 2$ is not as the atom $x \leq 3 \notin \Psi$.

It is well-known that LIA allows for quantifier elimination, that is, any LIA formula can be transformed into an equivalent quantifier-free formula [**?** **?**]. Let $\mathcal{U}$ be a subset $\{u_1, \cdots, u_n\}$ of variables. We use $\texttt{Exp}_{\mathcal{U}}$ for the set of expressions over $\mathcal{U}$. We use $\texttt{Form}_{\mathcal{U}}$ for the set of formulas of which free variables are $\mathcal{U}$. We use $\forall\mathcal{U}\phi$ for $\forall u_1 \cdots \forall u_n \phi$ and $\exists\mathcal{U}\phi$ for $\exists u_1 \cdots \exists u_n \phi$.

We say a set S of n-tuples of integers is *LIA-definable*, iff there is an LIA formula $\phi(x_1, \cdots, x_n)$ s.t. for every n-tuple $(z_1, \cdots, z_n)$ of integers, we have $(z_1, \cdots, z_n) \in S$ iff $\phi(z_1, \cdots, z_n)$ is valid. A set S of n-tuples of integers is *linear*, iff there are a finite set $\vec{z}_1, \cdots, \vec{z}_n$ of n-tuples of integers s.t. $\vec{z} \in S$ iff $\vec{z} + \lambda_1\vec{z}_1 + \cdots + \lambda_n\vec{z}_n \in S$ for all integers λ_i where $1 \leq i \leq n$. A set S of n-tuples of integers is *semi-linear*, iff it is a finite union of linear sets. **?**] showed that semi-linear sets exactly captures LIA-definable sets[1].

Theorem 1 ([**?**])**.** *A set S of n-tuples of integers is LIA-definable iff S is semi-linear.*

[1]The original theorems in [**?** **?**] is stated over the natural numbers. However, it can be extended to the integers.

?] established a fundamental connection between LIA-definable sets and periodicity of the set in one-dimensional case. A set S of integers is *ultimately periodic*, iff (1) there are two integers $n_l < 0$ and $p_l > 0$ s.t. for every $x \leq n_l$, $x \in S \cap \{z \mid z \leq n_l\}$ iff $x - p_l \in S \cap \{z \mid z \leq n_l\}$, and (2) there are two integers $n_u > 0$ and $p_u > 0$ s.t. for every $x \geq n_u$, $x \in S \cap \{z \mid z \geq n_u\}$ iff $x + p_u \in S \cap \{z \mid z \geq n_u\}$.

Theorem 2 ([**?**]). *A set S of integers is LIA-definable iff S is ultimately periodic.*

For example, the set of even integers has period 2 and is therefore LIA-definable, which can be represented by $x \equiv_2 0$. However, the set of prime numbers is not LIA-definable.

2.2. Counter machine

Definition 1 ([**?**]). A *n-counter machine*[2] M consists of

1. a set $(r_1, \cdots, r_n)$ of n count registers, each of which can hold a non-negative integer;
2. a special index register r_0 which denotes the label of current instruction to be executed;
3. a list L of labeled instructions, which comprises four types: `Halt`, `Inc`(i), `Dec`(i) and `JZ`(i, j).

A configuration C of a counter machine M is an interpretation that maps every register r into an positive integer $v(r)$. A configuration C is a *halting* configuration, iff $C(r_0) : \texttt{Halt} \in L$. The single step configuration $M(C)$ of M on C is defined as

[2]There are many variants of counter machines [**? ? ? ?**]. We adopt a simplified variant proposed in [**?**] which remains Turing-complete and thus preserves the undecidability of the halting problem.

$$M(C)(r) = \begin{cases} C(r_0)+1, & \text{if } v_0 : \texttt{Inc}(i) \in L \text{ and } r = r_0; \\ C(r_i)+1, & \text{if } v_0 : \texttt{Inc}(i) \in L \text{ and } r = r_i; \\ C(r_j), & \text{if } v_0 : \texttt{Inc}(i) \in L \text{ and } r_j \neq r_0 \text{ and } i \neq j; \\ C(r_0)+1, & \text{if } v_0 : \texttt{Dec}(i) \in L \text{ and } r = r_0; \\ C(r_i)-1, & \text{if } v_0 : \texttt{Dec}(i) \in L \text{ and } r = r_i; \\ C(r_j), & \text{if } v_0 : \texttt{Dec}(i) \in L \text{ and } r_j \neq r_0 \text{ and } i \neq j; \\ l, & \text{if } v_0 : \texttt{JZ}(i,l) \in L \text{ and } r = r_0 \text{ and } r_i = 0; \\ C(r_0)+1, & \text{if } v_0 : \texttt{JZ}(i,l) \in L \text{ and } r = r_0 \text{ and } r_i \neq 0; \\ C(r_j), & \text{if } v_0 : \texttt{JZ}(i,l) \in L \text{ and } r = r_j. \end{cases}$$

Intuitively, $\texttt{Inc}(i)$ increments the i-th count register and proceeds to the next instruction on L, while leaving all other registers unchanged. The instruction $\texttt{Dec}(i)$ behaves like $\texttt{Inc}(i)$ except it decrements the i-th count register. The instruction $\texttt{JZ}(i,l)$ tests whether the the i-th count register equals to 0. If so, it jumps to the l-th instruction; otherwise, it moves to the next one.

We use $M^i(C)$ to denote the i-th step configuration of M on C, which is defined as $M^1(C) = M(C)$ and $M(M^{i-1}(C))$ for $2 \leq i \leq n$. We say a counter machine M *terminating* in a configuration C, iff there is a run $C, M^1(C), \cdots, M^m(C)$ s.t. $M^m(C)$ is a halting configuration. The *pointwise halting problem* is to decide if M is terminating in a configuration C and the *universal halting problem* is to decide if M terminates on all configurations. It is well-known that the pointwise halting problem of n-counter machine for $n \geq 2$ is undecidable [**?**]. The undecidability result still hold for the universal halting problem [**?**].

2.3. Partial maximum satisfiability

Given a propositional formula in conjunctive normal form, the *satisfiability (SAT)* problem aims to determine if there is an assignment that satisfies all of the clauses in the formula. The *maximum satisfiability (MaxSAT)* problem, the

optimization variation of the SAT problem, searches a truth assignment that satisfies the maximum number of clauses. One of the significant generalizations of the MaxSAT problem is the *partial MaxSAT* problem. It divides the clauses into two types: *hard* and *soft*, and searches an assignment that satisfies all of the hard clauses and maximizes the number of satisfied soft clauses.

2.4. Impartial combinatorial games

In this section, we briefly introduce *impartial combinatorial games (ICGs)*. A game that satisfies the following conditions is called an ICG [**?**]: (1) There are two players and many states such that the player can move from one state to another one. (2) Two players alternate moving and have the same choice of moving. (3) The game ends when it moves to an *ending state* in which no player has a possible move. (4) The game always ends in a finite number of moves, and the last player to move wins. An *instance* of an ICG is the ICG together with an initial state. In general, an ICG may contain infinite states, and hence it may have infinite instances.

Each instance of an ICG is a finite two-player game with perfect information and no-chance move where the players take turns moving. In addition, this game does not end in a draw. By Zermelo's Theorem [**?**], there always exists a winning strategy for one player in an ICG.

It can be easily observed that the player cannot guarantee to win in all states. To distinguish states in which the player can forces to win from those in which she cannot, we classify states into two types: *winning* and *losing states*.

Definition 2 ([**?**])**.** In an ICG, the sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are recursively defined as

1. All ending states are losing states.
2. All states such that there is at least one move to a losing state are winning states.
3. All states such that the only possible moves are to winning states are losing states.

Definition 2 provides the exact characteristics of winning and losing states in a recursive way. In the base case, Condition 1 requires all legal ending states to be losing states. Both Conditions 2 and 3 are induction steps. Condition 2 says that every state such that at least one applicable move results in a losing state is a winning state. Condition 3 means that every state such that all possible successor states are winning states is a losing state. The sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are disjoint.

The above definition characterizes the concepts of losing states and winning states under the normal play rule. Under the misère rule, the definitions of winning and losing states are the same as Definition 2 except that all ending states are winning states. For the sake of clarity, throughout this paper, we assume that ICGs follow the normal play rule unless explicitly stated otherwise. Our approach to constructing winning strategies can be easily lifted to the case under the misère rule and will be discussed in Section 6.

We hereafter introduce the concept of *strategy*. ICGs are a special case of Gale-Stewart games, and so are determined by memoryless strategies [**?**]. In addition, it is impossible to obtain a winning strategy for the player in a losing state, there is no need to consider how to choose a move in this case. We hence define a *strategy* as a function from winning states to moves. A strategy is *winning*, iff it is a strategy and every winning state leads to a losing state following the move defined in this strategy.

3. A logical framework

In this section, we first introduce the concepts of numeric states and actions, and then present a logical framework for describing ICGs based on numeric states and actions. The framework we adopt is a slight modification of the linear integer arithmetic fragment of the PDDL 2.1 language defined in [**?**].

3.1. Numeric states and actions

Throughout this paper, we use the notion $\mathcal{X}$ for the set of state variables that are used to define numeric states. An expression e (resp. a formula ϕ)

is *semi-ground*, if $e \in \texttt{Exp}_{\mathcal{X}}$ (resp. $\phi \in \texttt{Form}_{\mathcal{X}}$). For example, suppose that $\mathcal{X} = \{v_1, v_2\}$. The expression $v_1 - 1$ is semi-ground, but $v_2 + k$ is not since k is not a state variable.

A *numeric state* s is an interpretation that maps every state variable v to an integer $v(s)$. For a state s, we can evaluate a semi-ground expression e into an integer $e(s)$ that the expression simplifies to when replacing each state variable v with its corresponding value $v(s)$. The Boolean value $\phi(s)$ of a semi-ground formula ϕ can be defined in a similar way. A numeric state s *satisfies* a formula ϕ, denoted by $s \models \phi$, iff $\phi(s) = \top$. A formula ϕ *implies* another one ψ, denoted by $\phi \models \psi$, iff $s \models \psi$ for every state s satisfying ϕ. A formula ϕ is *valid*, iff $s \models \phi$ for every state s; otherwise, it is *invalid*.

Each *numeric action* $a(\mathcal{K})$ is parameterized by a vector $\mathcal{K}$ of numeric variables, and is defined by a pair $\langle \texttt{pre}, \texttt{eff} \rangle$ where $\texttt{pre} \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$ denotes the precondition and $\texttt{eff}$ is the effect represented by a set of numeric effects. A *numeric effect* is a triple $\langle \phi, v, e \rangle$ where $\phi \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$, $v \in \mathcal{X}$, and $e \in \texttt{Exp}_{\mathcal{X} \cup \mathcal{K}}$. Intuitively, it means that if ϕ holds in the state s, then the value of v becomes $e(s)$ after executing the action; otherwise, it remains unchanged. Given an action $a(\mathcal{K})$, we use $\texttt{pre}(a(\mathcal{K}))$ and $\texttt{eff}(a(\mathcal{K}))$ for its precondition and effect, respectively.

Let $a(\mathcal{K})$ be an action and Σ a vector of semi-ground expressions where $\mathcal{K}$ and Σ have the same length n. The notation $e[\Sigma]$ (resp. $\phi[\Sigma]$) is the semi-ground expression (resp. formula) obtained from $e \in \exp_{\mathcal{X} \cup \mathcal{K}}$ (resp. $\phi \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$) by replacing the i-th parameter k_i of $\mathcal{K}$ with the i-th semi-ground expression e_i of Σ for $1 \leq i \leq n$. A *semi-ground action* $a[\Sigma]$ is defined as $\langle \texttt{pre}, \texttt{eff} \rangle$ where $\texttt{pre}(a[\Sigma]) = \texttt{pre}(a(\mathcal{K}))[\Sigma]$ and $\langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ for every $\langle \phi, v, e \rangle \in \texttt{eff}(a(\mathcal{K}))$. A semi-ground action $a[\Sigma]$ is *applicable* in a state s, iff $s \models \texttt{pre}(a[\Sigma])$. The result of applying an applicable action $a[\Sigma]$ in a state s is a successor state, written $\tau(a[\Sigma], s)$, where every variable v is associated to an integer $v(\tau(s, a[\Sigma]))$ defined as:

$$v(\tau(s, a[\Sigma])) = \begin{cases} e[\Sigma](s), & \text{if there is } \langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma]) \text{ and } \phi[\Sigma](s) = \top; \\ v(s), & \text{otherwise.} \end{cases}$$

We require that every action $a(\mathcal{K})$ is not *self-contradictory*, formally, for every state s and every vector Σ of semi-ground expressions, if $\langle \phi_1[\Sigma], v_1, e_1[\Sigma] \rangle$, $\langle \phi_2[\Sigma], v_2, e_2[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ s.t. $\phi_1[\Sigma](s) = \phi_2[\Sigma](s) = \top$, then $e_1[\Sigma](s) = e_2[\Sigma](s)$.

For brevity, we sometimes use a for (semi-ground) actions.

(Do we need ground actions ??)

3.2. Formalization

Definition 3. An LIA-definable game is defined as a tuple $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ where

- $\mathcal{X}$: a finite set of state variables.
- $\mathcal{A}$: a finite set of numeric actions.
- $\mathcal{C}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all legal states.
- $\mathcal{E}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all ending states.

Definition 3 is so general that it describes not only LIA-definable ICG but also other types of LIA-definable games that do not comply with the terminating condition. We present a non-terminating LIA-definable game. This game has a single variable v and a single action inc that is always applicable and that has the effect of incrementing the value of v. The set of ending states is the set of states where $v = 0$. This game starts with a state with $x > 0$ and goes on forever.

- $\mathcal{X} : \{v\}$;
- $\mathcal{A} : \{inc\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\texttt{pre}(inc) : \top$;
- $\texttt{eff}(inc) : \{\langle \top, v, v + 1 \rangle\}$.

The following theorem establishes the undecidability of deciding if an LIA-definable game is an LIA-definable ICG.

Theorem 3. *The problem deciding if an LIA-definable game is an LIA-definable ICG is undecidable.*

Proof. We can encode a 2-counter machine M in an LIA-definable game $\Pi(M)$ with 2 state variables x_1 and x_2 where x_1 encodes the index register and the first count register and x_2 denotes the second count register. Suppose that L is the list of labeled instructions in M and $|L|$ is the length of L. Let $n = \lceil \log(|L|) \rceil$. Since L is fixed for a 2-counter machine M, $|L|$ and n are fixed integers not variables. We use the lower n-bits of x_1 to represent the value of the index register and the remaining upper bits to represent the value of the first count register. The constraint $\mathcal{C}$ is $(\bigvee_{1 \leq l \leq |L|} x_1 \equiv_n l) \wedge x_1 > 0 \wedge x_2 \geq 0$. The constraint ensures that (1) the value of index register is in the feasible range (*i.e.*, $r_0 \in [1, |L|]$) in every legal state; and (2) the value of each count register is a natural number. The ending condition $\mathcal{E}$ is $\bigvee_{l:\texttt{Halt} \in L} x_1 \equiv_n l$, meaning that the execution of instructions ends in instruction `Halt`. Each labeled non-`Halt` instruction is stimulated by an action a as follows:

1. $\texttt{pre}(a) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a) : \{\langle \top, x_1, x_1 + 2^n + 1 \rangle\}$ for an instruction of the form $l : \texttt{Inc}(1)$;
2. $\texttt{pre}(a) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a) : \{\langle \top, x_1, x_1 + 1 \rangle, \langle \top, x_2, x_2 + 1 \rangle\}$ for an instruction of the form $l : \texttt{Inc}(2)$;
3. $\texttt{pre}(a) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a) : \{\langle \top, x_1, x_1 - 2^n + 1 \rangle\}$ for an instruction of the form $l : \texttt{Dec}(1)$;
4. $\texttt{pre}(a) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a) : \{\langle \top, x_1, x_1 + 1 \rangle, \langle \top, x_2, x_2 - 1 \rangle\}$ for an instruction of the form $l : \texttt{Dec}(2)$;
5. $\texttt{pre}(a(k)) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a(k)) : \{\langle x_1 = l, x_1, x_1 + 1 \rangle, \langle x_1 \neq l, x_1, x_1 - l + k \rangle\}$ for an instruction of the form $l : \texttt{JZ}(1, k)$;
6. $\texttt{pre}(a(k)) : x_1 \equiv_{2^n} l$ and $\texttt{eff}(a(k)) : \{\langle x_2 = 0, x_1, x_1 + 1 \rangle, \langle x_2 \neq 0, x_1, k \rangle\}$ for an instruction of the form $l : \texttt{JZ}(2, k)$.

It is easily verified that M halts at all configurations iff $\Pi(M)$ ends in a finite number of moves. Since the universal halting problem of 2-counter machine is undecidable, deciding if an LIA-definable game is an LIA-definable ICG is also undecidable. □

To avoid confusion, in the rest of this paper, we require every game formalized according to Definition 3 be an ICG.

To obtain the finite representation of winning states, we use an arithmetic formula to exactly capture this notion, called *the winning formula*.

Definition 4. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. We say a formula $\phi \in \texttt{Form}_{\mathcal{X}}$ is a *winning formula* of Π, iff for every state s, $s \models \mathcal{C} \wedge \phi$ iff s is a legal winning state.

The winning formula ϕ along with the formula $\mathcal{C}$ exactly characterizes the set of winning states. In general, there are many logical different winning formulas for an ICG. But they are logically equivalent under the background theory $\mathcal{C}$, that is, $\mathcal{C} \models \phi \equiv \phi'$. Hence, we sometimes call a winning formula the winning one. Since the sets of winning states and losing states are disjoint, $\neg\phi$ is the symbolic representation of losing states. As proved in [**?**], the winning formulas of certain games are not LIA-definable. In this paper, we focus on the synthesis of LIA-definable winning formulas and leave the synthesis of winning formulas that require more expressive languages to future work.

A *rule* (ψ, a) is a pair of semi-ground formulas ψ and semi-ground actions a. Given a rule (ψ, a), we call ψ the *condition formula* of (ψ, a). A *strategy* is a set of rules.

Definition 5. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- A rule (ψ, a) is *winning*, iff for every state s satisfying $\mathcal{C} \wedge \phi \wedge \psi$, we have a is applicable in s and $\tau(s, a)$ is a losing state.
- A rule (ψ, a) is *complete*, iff for every state s s.t. $\tau(s, a)$ is a losing state, we have $\psi(s) = \top$.

- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *winning*, iff every rule (ψ_i, a_i) is winning.
- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *complete*, iff $\bigvee_{i=1}^{n} \psi_i$ is the winning formula.

The winning property of the rule (ψ, a) requires that executing the action a in every state satisfying $\mathcal{C} \wedge \phi \wedge \psi$ yields a losing state. The completeness property of the rule says that the formula ψ covers all of the states s s.t. its successor state $\tau(s, a)$ is a losing state. The winning property of the strategy means that each of its rule is a winning rule. The completeness property of the strategy stipulates that the disjunction of the condition formula ψ_i of each rule is the winning formula.

Example 2. The formalization of the Take-away-3 game [**?**] is given as follows. The state variable v denotes the number of the remaining chips in this game. For simplicity, we use an integer c to denote the numeric state where v is assigned to c. A move consists of taking any number (from 1 to m) of chips. Here we consider the case where $m = 3$. The following are the definition of ICG and the description of actions.

- $\mathcal{X} : \{v\}$;
- $\mathcal{A} : \{take(k)\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\texttt{pre}(take(k)) : v \geq k \wedge k > 0 \wedge k \leq 3$;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$.

The two numeric states 0 and 4 are legal losing states. The former is a legal ending state. The latter has three applicable semi-ground actions: $take(1)$, $take(2)$, and $take(3)$. Executing each of them yields three different winning states $\tau(4, take(1)) = 3$, $\tau(4, take(2)) = 2$, and $\tau(4, take(3)) = 1$.

Algorithm 1: `SynWinForm`

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an ICG.

Output: ϕ: the winnig formula of Π

```
1 Initialize the set S of legal states
2 P, N ← ClassifyStates(S)
3 while true do
4     φ ← GenerateCandidate(P, N)
5     S' ← VerifyWinForm(φ)
6     if S' = ∅ then return φ ;
7     else
8         P', N' ← ClassifyStates(S')
9         P ← P ∪ P' and N ← N ∪ N'
```

The winning formula of this game is $v \not\equiv_4 0$. It means that any legal state in which the number of the remaining chips is not a multiple of 4 is a winning state. The three rules $(v \equiv_4 1, take(1)), (v \equiv_4 2, take(2))$ and $(v \equiv_4 3, take(3))$ are winning and complete rules, and form a winning and complete strategy. But $(v = 2, take(2))$ is a winning but incomplete rule as there is a state 6 from which taking 2 chips leads to a losing state, which does not satisfy $v = 2$. □

4. Synthesizing winning formulas

In this section, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas for ICGs over infinite states, illustrated in Algorithm 1. A counterexample in this approach in fact is a state in ICGs.

4.1. The main algorithm

The main idea of Algorithm 1 is as follows: It first attempts to generate a candidate formula ϕ via a set of counterexamples, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the

$\mathcal{P}$	$\mathcal{N}$	Candidate formulas	Counterexamples
1	0	$v = 1$	3
1, 3	0	$v > 0$	4
1, 3	0, 4	$v \not\equiv_2 0$	2
1, 2, 3	0, 4	$v \not\equiv_4 0$	$\emptyset$

Table 1: A run of synthesizing the winning formula

winning formula ϕ. Otherwise, a set of new counterexamples will be gathered and used to create a new candidate formula. The above steps repeat until the algorithm discovers the winning formula of the ICG.

We hereafter elaborate Algorithm 1 in details. It initializes the set S as a finite set of legal states, which is accomplished by an SMT solver (Line 1). Each legal state s is the state satisfying the formula $\mathcal{C}$ denoting all legal states (cf. Definition 3). The SMT solver takes the formula $\mathcal{C}$ as input, and returns a state satisfying $\mathcal{C}$. However, the SMT solver often returns the same state for the formula $\mathcal{C}$. We can easily fix this defect and generate many different legal states via adding an extra subformula on $\mathcal{C}$. Suppose that we are given a set S' of legal states. The formula $\mathcal{C}_{S'}$ is $\mathcal{C} \wedge \bigwedge_{s \in S'} \neg[\bigwedge_{v \in \mathcal{X}} (v = v(s))]$, and hence any state satisfying $\mathcal{C}_{S'}$ is a legal state not in S'.

The set S of states will be classified into two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states (Line 2). In an ICG, each positive state is a winning state while each negative state is a losing state.

Algorithm 1 then enters a loop until a winning formula is discovered (Lines 3 - 10). With the two sets of positive states and of negative states in hand, we can produce a candidate winning formula ϕ (Line 4). Then, we will check whether the candidate formula ϕ is a winning formula (Line 5). If ϕ is correct (that is, $S' = \emptyset$), then Algorithm 1 terminates with the solution ϕ (Line 7). Otherwise, the verification procedure yields some states of S' that are witnesses of its failure and incorporates them into $\mathcal{P}$ and $\mathcal{N}$, respectively (Lines 9 - 11).

Example 3. Table 1 shows the run of synthesizing the winning formula of

Take-away-3 game. At the beginning, only state 1 is in the set $\mathcal{P}$ and state 0 in $\mathcal{N}$. Our approach generates the formula $v = 1$ as the candidate solution that is consistent with $\mathcal{P}$ and $\mathcal{N}$. However, it is not the correct winning formula that holds for all the legal states and the SMT solver returns a winning state 3. Our approach takes into account the new state. Two candidate formulas $v > 0$ and $v \not\equiv_2 0$ are sequentially found. The above two formulas, however, both fail to verify and return a losing state 4 and a winning state 2, respectively, as counterexamples to the two formulas. Finally, the algorithm terminates with the correct winning formula $v \not\equiv_4 0$ that constructed from the above 5 states. □

Finally, three subprocedures of Algorithm 1 remain to be addressed: (1) classifying winning states from losing states (Line 2), (2) generating a candidate formula (Line 4), and (3) verifying the correctness of a candidate formula (Line 5). These will be resolved in the subsequent three sections.

4.2. Classification of numeric states

In this subsection, we design two algorithms to the problem of classifying states into winning and losing states. Both algorithms work in a recursive way based on Definition 2.

We first provide the notion of *transition formulas* and *global transition formulas*. The definition of transition formulas is a numerical generalization of the existing propositional encoding for the planning problem which focuses on transition systems over finite states [**?** **?**]. The transition formula $\mathcal{T}(a(\mathcal{K}))$ for an action $a(\mathcal{K})$ reflects the mapping from predecessor states to successor states due to the effect of $a(\mathcal{K})$. To distinguish the predecessor state variable and the successor one, we use the unprimed variable v for the former and the primed one v' for the latter.

Definition 6. The transition formula $\mathcal{T}(a(\mathcal{K}))$ of $a(\mathcal{K})$ is the conjunction of the following formulas:

1. $\texttt{pre}(a(\mathcal{K}))$;
2. $\bigwedge_{\langle \phi, v, e\rangle \in \texttt{eff}(a(\mathcal{K}))}(\phi \rightarrow v' = e)$;

3. $\bigwedge_{v \in \mathcal{V}}[(\bigvee_{\langle \phi, v, e \rangle \in \texttt{eff}(a(\mathcal{K}))} \phi) \vee v' = v]$.

The first formula is the precondition of action $a(\mathcal{K})$. The second one is the effect axiom meaning that the value of v becomes $e(s)$ if the condition ϕ holds in the state s. The third formula is the frame axiom meaning that if no condition ϕ s.t. $\langle \phi, v, e \rangle \in \texttt{eff}(a)$ holds, then the value of v remains unchanged.

The global transition formula $\mathcal{T}(\mathcal{A})$ is the union of the mapping due to the effect of every action $a(\mathcal{K}) \in \mathcal{A}$, denoting the state transition system of the whole game. The definition is as follows:

Definition 7. Let $\mathcal{A}$ be a set of actions. The global transition formula $\mathcal{T}(\mathcal{A})$ of Π is $\bigvee_{a(\mathcal{K}) \in \mathcal{A}} \exists \mathcal{K}[\mathcal{T}(a(\mathcal{K}))]$.

The following example illustrates how transition formulas are constructed from the definition of actions.

Example 4. Let us continue the Take-away-3 game. We construct the transition formula $\mathcal{T}(take(k))$ as follows. The precondition is $v \geq k \wedge k > 0 \wedge k \leq 3$. The effects contain one elements: $\langle \top, v, v - k \rangle$. Hence, the effect axiom of $\mathcal{T}(take(k))$ is $(\top \rightarrow v'_1 = v - k)$. We now consider the frame axiom of $\mathcal{T}(take(k))$. For variable v, the formula is $\top \vee v'_1 = v$. Conjoining the above formulas and simplifying the conjunction lead to the transition formula:

$$\mathcal{T}(take(k)) \equiv v \geq k \wedge k > 0 \wedge k \leq 3 \wedge v'_1 = v - k.$$

By Definition 7, the global transition formula

$$\mathcal{T}(\mathcal{A}) \equiv \exists k[\mathcal{T}(take(k))].$$

□

Algorithm 2 provides a recursive procedure for determining whether a legal state s in an LIA-definable ICG is a winning state. The algorithm employs memorization to avoid redundant computation by first checking if the result for state s has already been cached (Line 1). It then verifies if s is a legal ending state (Lines 2 - 4). If it is the case, then s is classified as a losing states since

Algorithm 2: `IsWinningState`(Π, ϕ, s)

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of Π;

s: a legal state.

Output: $\top$, if s is a winning state; $\bot$, otherwise.

```
1  if Cache(s) ≠ nil then return Cache(s) ;
2  if IsValid(φ(s) → C ∧ E) then
3      Cache(s) ← ⊥
4      return ⊥
5  foreach s' s.t. φ(s')[X/X'] ⊨ ∃X[T(A) ∧ φ(s)] do
6      if IsWinningState(Π, φ, s') = ⊥ then
7          Cache(s) ← ⊤
8          return ⊤
9  Cache(s) ← ⊥
10 return ⊥
```

all ending states are losing states (cf. Definition 2). Otherwise, Algorithm 2 iterates over every legal successor state s' of s (Lines 5 - 8). If at least one successor state s' is a losing state, then s is a winning state since all states such that there is at least one move to a losing state are winning states. Conversely, s is a losing state since all states such that the only possible moves are to winning states are losing states (Lines 9 - 10).

The following example illustrates the run of Algorithm 2.

Example 5. Let us consider the Take-away-3 game, starting from state 4. Figure 1 shows the run of Algorithm 2 for state 4. Since the value of state 4 has not been computed and it is not an ending state, Algorithm 2 iterates through its successor states 1, 2 and 3. State 1, also not previously computed and not an ending state, leads to its unique successor state 0. As state 0 is an ending state, it is classified as losing and stored in the cache. It follows that state 1 is a winning

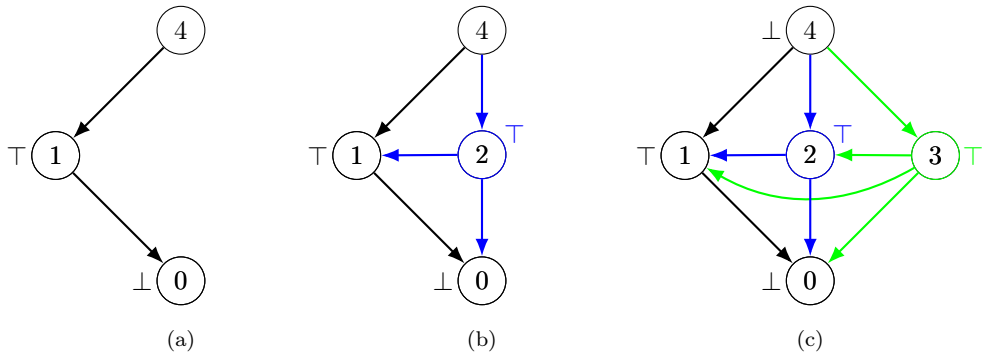


Figure 1: The run of Algorithm 2 in Take-away-3 game.

Algorithm 3: `IsWinningState'`(Π, ϕ, s)

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of Π;

s: a legal state.

Output: $\top$, if s is a winning state; $\bot$, otherwise.

$\psi \leftarrow (\mathcal{C} \wedge \mathcal{E})$ and $\varphi \leftarrow \bot$

if `IsValid`$(\phi(s) \rightarrow \psi)$ **then return** $\bot$;

else

  while true do

    $\varphi \leftarrow \varphi \vee \{\exists \mathcal{X}[\mathcal{T}(\mathcal{A}) \wedge \psi]\}[\mathcal{X}'/\mathcal{X}]$

    if `IsValid`$(\phi(s) \rightarrow \mathcal{C} \wedge \varphi)$ **then return** $\top$;

    $\psi \leftarrow \psi \vee \{\forall \mathcal{X}[\mathcal{T}(\mathcal{A}) \rightarrow \varphi]\}[\mathcal{X}'/\mathcal{X}]$

    if `IsValid`$(\phi(s) \rightarrow \mathcal{C} \wedge \psi)$ **then return** $\bot$;

state, which is stored in the cache (Figure 1(a)). Then, Algorithm 2 evaluates state 2 by examining its successor states 0 and 1. Since state 0 was already known as a losing state according to the memorization, state 2 is winning (Figure 1(c)). Similarly, state 3 is identified as winning. Because all of its successor states of state 4 are winning, state 4 itself is a losing state (Figure 1(d)). □

Algorithm 2 provides an efficient method to classifying numeric states. How-

ever, this algorithm faces a fundamental limitation when dealing with ICGs where a state have infinitely many successor states. The loop iterating through all successor states (Line 5) will not terminate in such case. Consequently, Algorithm 2 cannot serve as a terminating procedure for the decidability of determining if a state is winning for any LIA-definable ICG. To address this, we design another algorithm to check if any legal state is winning.

To distinguish state variables with different time point, we use $\mathcal{X}^i$ for $i = 0, 1 \cdots$ for the set of state variables at time-point i. For example, a state variable $v \in \mathcal{X}$ is denoted by v_0 at any ending state and by v_1 at a state that moves to an ending state via performing a single action. Given a formula ψ, $\psi[\mathcal{X}/\mathcal{X}^i, \mathcal{X}'/\mathcal{X}^j]$ is the formula obtained by replacing every occurrence of $v \in \mathcal{X}$ and $v' \in \mathcal{X}'$ in ψ with v^i and v^j, respectively. The notation of $\psi[\mathcal{X}/\mathcal{X}^i]$ is similarly defined.

Algorithm 3 proceeds by backward induction according to the definition of winning and losing states (cf. Definition 2). If state s is a winning state, then it returns $\top$; otherwise, it returns $\bot$. We iteratively constructs two formulas: ψ and φ. The former formula ψ represents the set of losing states where either all of them are ending states (Line 1) or they all have at least one successor state satisfies φ (Line 9). The latter formula φ denotes the set of winning states whose successor states all satisfy ψ (Line 6). We use $\phi(s)$ for the formula that denotes state s (more formally, $\phi(s) = \bigwedge_{v \in \mathcal{X}} v = v(s)$). If s satisfies ψ, then s is a losing state (Lines 2, 3, 7 and 8). Similarly, s is a winning state when s satisfies φ (Lines 10 and 11). Since any ICG always ends in a finite number of moves, the loop (Lines 5 - 11) is guaranteed to terminate.

We obtain the decidability of determining if a state is winning for any LIA-definable ICG.

Theorem 4. *The problem deciding if a state is winning for an LIA-definable ICG is decidable.*

We make a comparison between Algorithms 2 and 3. Algorithms 2 and 3 employ fundamentally different strategies to solve the same problem. In terms of search direction and mechanism, Algorithm 2 utilizes a practical forward

search approach. It starts from a given state s and explores its successors, using a cache to store results and avoid redundant computations. In contrast, Algorithm 3 adopts a symbolic backward induction method. Instead of explicit state traversal, it iteratively computes the entire sets of winning and losing states by constructing and solving quantified LIA formulas. In terms of computational cost and efficiency, Algorithm 2 is generally more efficient in practice because it directly explores the state space and reuses previously computed results. The computational cost of Algorithm 3 is typically higher than that of Algorithm 2, as constructing complex quantified formulas and repeatedly invoking an SMT solver is often computationally intensive. In terms of generality and theoretical proof, Algorithm 2 cannot serve as a general decision procedure for all LIA-definable ICGs, as its reliance on iterating through successors makes it unable to handle cases where a state has infinitely many successors. Conversely, the symbolic approach of Algorithm 3 allows it to handle games with infinite successors, thereby providing a theoretical proof of decidability. To summarize, Algorithm 2 is an efficient but non-general algorithm, whereas Algorithm 3 is a general algorithm designed for theoretical completeness, albeit with a higher computational cost.

4.3. Generation of candidate formulas

In this subsection, we propose two approaches for generating candidate formulas. The first approach adopts a simple enumerative algorithm proposed in [**?**]. The second approach formulate the task as exact binary classification problem and encode each classification problem as an instance of the partial MaxSAT problem.

4.3.1. Enumeration-based approach

The key to the enumeration algorithm is to process generate the candidate formulas (or expressions) by induction on size according to the grammar illustrated in Section 2.1. In the first step, only arithmetic expressions of size 1 (*i.e.*, state variables and integers) are considered. As the set of integers is infinite, the

integers considered in the first step are 0, 1 together with the constants explicitly occurring in the formalization of games. Firstly, This restriction does not limit the coverage of all integers since any positive integer c can be expressed as the sum of c 1's, while the negative integer $-c$ equals to $0 - c$. Furthermore, we observe that some integers appearing in the winning formula and strategy are close to some explicit constants defined in the formalization. Hence, supplementing these constants can accelerate the synthesis process. In the k-th step, we generate arithmetic expressions and formulas of size k. The enumeration process terminates when it finds candidates formula ϕ such that $\{s \mid s \models \phi \wedge \mathcal{C}\} = \mathcal{P}$.

To improve the efficiency of the enumeration process, we remove expressions and formulas based on the notion of indistinguishability. Formally, two formulas ϕ_1 and ϕ_2 are *indistinguishable* w.r.t. a set S of states, if $\phi_1(s) = \phi_2(s)$ for every $s \in S$. The definition of indistinguishability between two expressions is similar. For example, suppose that $S = \{(v_1 : 1, v_2 : 2), (v_1 : 2, v_2 : 2)\}$. It is easily verified that the two formulas $v_1 \neq v_2$ and $v_1 + 1 = v_2$ hold in the first state and does not hold in the second one. Therefore, the two formulas are indistinguishable w.r.t. S. Suppose that we are generating expressions and formulas of size k. If ϕ_1 and ϕ_2 are indistinguishable w.r.t. S, then it is sufficient to carry forward only one of ϕ_1 and ϕ_2 to the next iteration $k + 1$. Based on the notion of indistinguishability, we can prune the search space of candidate formulas.

4.3.2. Exact binary classification-based approach

We observe that enumeration-based approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula increases. To accelerate the generation process, we introduce the exact binary classification problem and propose an approach that encodes this problem as a partial MaxSAT problem.

In contrast to enumeration-based approach, we firstly generates a set Ψ of arithmetic atoms rather than formulas of increasing sizes until for every pair of positive state and negative state, there is an atom $\psi \in \Psi$ classifying them. In order to prune the huge size of Ψ, we also eliminate some arithmetic atoms

in Ψ based on indistinguishability. Based on the set Ψ of arithmetic atoms, we reduce the problem of searching for candidate formulas to the exact binary classification problem.

Definition 8. Given two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and a set Ψ of arithmetic atoms, the task of the exact binary classification problem is to generate a formula ϕ based on Ψ s.t. $\phi(s) = \top$ for every state $s \in \mathcal{P}$ and $\phi(s) = \bot$ for every state $s \in \mathcal{N}$.

The exact binary classification problem may not have a solution in some cases. The sufficient and necessary condition for the existence of a solution is established by the following theorem.

Theorem 5. *Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The following two statements are equivalent:*

1. *For each pair of positive state s and negative state t, there is an atom $\psi \in \Psi$ s.t. $\psi(s) \neq \psi(t)$.*
2. *There exists a formula ϕ based on Ψ that classifies $\mathcal{P}$ from $\mathcal{N}$.*

To prove the above theorem, we need the following two definitions that are used to synthesize a desired formula for the exact binary classification problem if the solution exists.

Definition 9. Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The *filter set* $\Psi_{\mathcal{P},\mathcal{N}}$ of Ψ w.r.t. $\mathcal{P}$ and $\mathcal{N}$ is $\{\psi \mid \psi \in \Psi \text{ and } \psi(s) \neq \psi(t) \text{ for some } s \in \mathcal{P} \text{ and } t \in \mathcal{N}\}$.

The filter set is the subset of Ψ in which each element classifies at least one positive state and one negative state.

Definition 10. Let S be a set of states and Ψ a set of atoms. The *characteristic formula* of S based on Ψ is $\bigvee_{s \in S} \left[\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg\psi \right) \right]$.

The conjunct $\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg\psi \right)$ can be considered as a truth assignment of the state s on the set Ψ of atoms. The characteristic formula collects the assignments of every state $s \in S$ on Ψ.

Proof of Theorem 5: ($\Leftarrow$): Let ϕ be a formula based on Ψ s.t. ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. We prove the first statement by contradiction. Let $s \in \mathcal{P}$ and $t \in \mathcal{N}$ s.t. no atom $\psi \in \Psi$ satisfies the property $\psi(s) \neq \psi(t)$. It follows that $\phi(s) = \phi(t)$. This contradicts the assumption.

($\Rightarrow$): Let ϕ be the characteristic formula of $\mathcal{P}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$. Clearly, ϕ is also based on Ψ. By Definition 10, for every positive state s, ϕ contains $\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s)=\top} \psi\right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s)=\bot} \neg\psi\right)$. Hence, $s \models \phi$. By the first statement of Theorem 5 and Definition 9, for every negative state t, there is an atom $\psi \in \Psi_{\mathcal{P},\mathcal{N}}$ and a positive state s s.t. $\psi(t) \neq \psi(s)$. None of the conjuncts of ϕ is satisfied by t. Hence, $t \not\models \phi$. In summary, ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. □

The proof of Theorem 5 offers a direct method to obtain a formula that discriminates positive states from negative ones. However, the given set Ψ in general includes a large number of arithmetic atoms. So is the filter set $\Psi_{\mathcal{P},\mathcal{N}}$. Therefore, the characteristic formula of $\mathcal{P}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$ has large size and is difficult to be applied in generating candidate formulas.

To decrease the amount of atoms in the solution, we devise a practical method to the exact binary classification problem based on partial MaxSAT. The propositional encoding $T(\mathcal{P}, \mathcal{N}, \Psi)$ takes as inputs two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and the set Ψ of arithmetic atoms. The propositions in $T(\mathcal{P}, \mathcal{N}, \Psi)$ are

- $Sel(\psi)$: the atom $\psi \in \Psi$ appears in the formula ϕ.

The hard constraints of $T(\mathcal{P}, \mathcal{N}, \Psi)$ are

- $\bigvee_{\psi(s) \neq \psi(t)} Sel(\psi)$ for every $s \in \mathcal{P}$ and $t \in \mathcal{N}$.

It means that for every pair of positive state s and negative state t, at least one atom of Ψ that distinguishes between s and t is chosen, corresponding to the first statement of Theorem 5.

The clauses in the soft constraints of $T(\mathcal{P}, \mathcal{N}, \Psi)$ are the negative literal $\neg Sel(\psi)$ for each atom ψ. The partial MaxSAT solver looks for a solution with the fewest possible arithmetic atoms.

States		$v = 0$	$v \equiv_2 0$	$v \equiv_3 0$	$v \equiv_4 0$
$\mathcal{P}$	1	$\bot$	$\bot$	$\bot$	$\bot$
	2	$\bot$	$\top$	$\bot$	$\bot$
	3	$\bot$	$\bot$	$\top$	$\bot$
$\mathcal{N}$	0	$\top$	$\top$	$\top$	$\top$
	4	$\bot$	$\top$	$\bot$	$\top$

Table 2: The example of learning the characteristic formula

Example 6. We use Take-away-3 game to illustrate the method to learn the characteristic formula. Suppose that we are given the sets of positive states $\mathcal{P} : \{1, 2, 3\}$ and of negative states $\mathcal{N} : \{0, 4\}$, and the set of arithmetic atoms $\Psi : \{v = 0, v \equiv_2 0, v \equiv_3 0, v \equiv_4 0\}$. The Boolean value of every atom on every state is shown in Table 2. The propositional encoding $T(\mathcal{P}, \mathcal{N}, \Psi)$ include four propositions: $Sel(v = 0)$, $Sel(v \equiv_2 0)$, $Sel(v \equiv_3 0)$ and $Sel(v \equiv_4 0)$. Since four atoms distinguish positive state 1 and negative state 0, the hard constraints for 1 and 0 is $Sel(v = 0) \vee Sel(v \equiv_2 0) \vee Sel(v \equiv_3 0) \vee Sel(v \equiv_4 0)$. The hard constraints for other pairs of positive state and negative state can be similarly constructed. The soft constraints are the negation of each proposition: $\neg Sel(v = 0)$, $\neg Sel(v \equiv_2 0)$, $\neg Sel(v \equiv_3 0)$ and $\neg Sel(v \equiv_4 0)$. The partial MaxSAT solver returns the following assignment:

$$Sel(v = 0) = \bot, Sel(v \equiv_2 0) = \bot, Sel(v \equiv_3 0) = \bot, Sel(v \equiv_4 0) = \top.$$

Only the atom $v \equiv_4 0$ is picked from the assignment. As the chosen atom does not hold in every state of $\mathcal{P}$, its negation $v \not\equiv_4 0$ is the characteristic formula of $\mathcal{P}$ based on Ψ. In addition, it exactly classifies the given states of positive states and of negative states. □

4.4. Correctness verification

In this subsection, we present the constraints for the winning formula that is used to verifying the correctness of candidate formula based on the notion of global transition formulas (cf. Definition 7).

Definition 11. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The constraints for the winning formula ϕ of Π are as follows:

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$;
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$;
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$.

where $\psi[\mathcal{X}/\mathcal{X}']$ is the formula obtained by replacing every occurrence of $v \in \mathcal{X}$ in ψ with v'.

Each of the above constraints corresponds to each condition of winning and losing states (cf. Definition 2). As a result, we obtain the soundness theorem of the constraints for the winning formula.

Theorem 6. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. A formula ϕ is the winning formula of Π iff all of the constraints illustrated in Definition 11 are valid.*

Proof. ($\Rightarrow$): Assume that ϕ is the winning formula. It follows that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing. We prove the three constraints illustrated in Definition 11 are valid.

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$: Let s be a state satisfying $\mathcal{C} \wedge \mathcal{E}$. It follows that s is a legal ending state. By the first item of the definition of winning and losing states (cf. Definition 2), s is a losing state. Hence, $s \models \neg\phi$ and the first constraint is valid.
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$: Let s be a state satisfying $\mathcal{C} \wedge \phi$. It follows that s is a legal winning state. By the second item of the definition of winning and losing states, there is at least one applicable action a in s s.t. $\tau(s, a)$ is a losing state . Hence, $s \models \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$ and the second constraint is valid.
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$: The proof of the validity of the third constraint is the same as that of the second constraint by using the third item of the definition of winning and losing states.

($\Leftarrow$): Assume that ϕ is a formula s.t. all of the constraints illustrated in Definition 11 is valid. We prove that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing by induction on s.

Base case (s is a legal ending state): It follows that $s \models \mathcal{C} \wedge \mathcal{E}$. By the first constraint $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$, we get that $s \models \neg\phi$. By the definition of winning formula (cf. Definition 4), s is a losing state.

Inductive step (s is a legal but not ending state): Suppose that $s \models \phi$. By the second constraint $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$, there is a successor state $\tau(s, a)$ of s s.t. $\tau(s, a) \models \mathcal{C} \wedge \neg\phi$. By the inductive assumption, $\tau(s, a)$ is a losing state. By the second item of the definition of winning and losing states, s is a winning state. Similarly, we can prove that s is a losing state when $s \models \neg\phi$ by using the third constraint $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$ and the third item of the definition of winning and losing states. □

Theorem 6 reduces the correctness verification problem of an LIA-formula for an LIA-definable ICG to the validity problem of an LIA-formula. Thanks to the decidability of the validity problem of LIA [**?**], the above correctness verification problem is decidable.

Corollary 1. *The problem deciding if an LIA-formula is a winning formula for an LIA-definable ICG is decidable.*

In addition, Theorem 6 confirms the decidability of correctness verification of an LIA-formula and the soundness property of Algorithm 1 for synthesizing the winning formula.

Corollary 2. *Let* $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ *be an ICG. The formula returned by Algorithm 1 is the winning formula of* Π.

Proof sketch: Lines 5 of Algorithm 1 checks if candidate formula ϕ is the winning formula according to the constraints illustrated in Definition 11. If $S' = \emptyset$, then the constraints for the winning formula are valid. By Theorem 6, ϕ is the winning formula. □

5. Synthesizing winning strategies

In this section, we also devise a CEGIS approach to synthesizing winning strategies. We first introduce the main algorithm, illustrated in Algorithm 4, and followed by correctness verification of winning strategies.

5.1. The main algorithm

The main synthesis algorithm first generate a pool of candidate actions for the set of positive states. For each action, we synthesize its condition formula via the candidate formula generation approach. This step will continue until we synthesize a winning strategy that consists of a set of pairs of action and its condition formula. The following gives a comprehensive explanation of Algorithm 4.

After the process of synthesizing winning formulas, we obtain the winning formula ϕ along with two sets $\mathcal{P}$ of winning states and $\mathcal{N}$ of losing states, which serve as inputs to Algorithm 4. In the main loop of Algorithm 4, it first generates a pool $\mathcal{A}_p$ of semi-ground actions such that every state $s \in \mathcal{P}$ has at least one action $a \in \mathcal{A}_p$ that transforms s to a losing state $\tau(s, a)$ (Line 3). For every action a of $\mathcal{A}_p$, if it does not appear in the strategy δ, then the inner loop of Algorithm 4 attempts to generate a formula ψ_a and produces a rule (ψ_a, a) (Lines 4 - 17). We generate the formula ϕ_a via candidate formula generation approach, which was introduced in Section 4.3. We then collect the set $\mathcal{P}_a$ of positive states and $\mathcal{N}_a$ of negative states for this problem (Lines 5 - 11). A state $s \in \mathcal{P}$ is a positive state for generating the formula ϕ_a, if the following conditions hold: (1) the action a is applicable over s; and (2) the successor state $\tau(s, a)$ is a losing state. In contrast, the set $\mathcal{N}_a$ includes the positive states of $\mathcal{P}$ not comply with the above two conditions and all negative states of $\mathcal{N}$. Similarly to Algorithm 1, it creates the candidate formula ψ_a that classifies $\mathcal{P}_a$ from $\mathcal{N}_a$.

The correctness of the rule (ψ_a, a) will be validated. If the rule (ψ_a, a) is winning, then no counterexample is returned by the SMT solver (Lines 14 and 15). Otherwise, we are given a set S' of states certifying that ψ_a is not a desired formula for the action a. Each state $s \in S'$ enjoy the following two properties:

Algorithm 4: $\texttt{SynWinStrat}(\phi, \mathcal{P}, \mathcal{N})$

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, z, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of the Π;

$\mathcal{P}$: the set of positive states;

$\mathcal{N}$: the set of negative states.

Output: δ: a winning and complete strategy

$\delta^c, \delta^w, \mathcal{A}^f, \mathcal{A}^u \leftarrow \emptyset$

$\mathcal{P}' \leftarrow \mathcal{P}$

while true do

  $\mathcal{P}'' \leftarrow \mathcal{P}'$ and $\mathcal{P}' \leftarrow \emptyset$

  $\mathcal{A}' \leftarrow \texttt{EnumerateActions}(\mathcal{P}'') \setminus \mathcal{A}^f$

  foreach $a \in \mathcal{A}^u \cup \mathcal{A}'$ **do**

    if δ^c *does not cover* a **then**

      $\mathcal{A}^f \leftarrow \mathcal{A}^f \cup \{a\}$ and $\mathcal{A}^u \leftarrow \mathcal{A}^u \setminus \{a\}$

      continue;

    if $a \notin \mathcal{A}^u$ **then**

      $\mathcal{P}_a \leftarrow \{s \in \mathcal{P} \mid s \models \texttt{pre}(a) \text{ and } \tau(s,a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N} \cup \{s \in \mathcal{P} \mid s \not\models \texttt{pre}(a) \text{ or } \tau(s,a) \notin \mathcal{L}\}$

    else /* $a \in \mathcal{A}^u$ */

      $\mathcal{P}_a \leftarrow \mathcal{P}_a \cup \{s \in \mathcal{P}'' \mid s \models \texttt{pre}(a) \text{ and } \tau(s,a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N}_a \cup \{s \in \mathcal{P}'' \mid s \not\models \texttt{pre}(a) \text{ or } \tau(s,a) \notin \mathcal{L}\}$

    $\psi_a \leftarrow \texttt{GenerateCandidate}(\mathcal{P}_a, \mathcal{N}_a)$

    $S_a^w \leftarrow \texttt{VerifyWinningRule}(\psi_a, a)$

    if $S_a^w = \emptyset$ **then**

      $S_a^c \leftarrow \texttt{VerifyCompleteRule}(\psi_a, a)$

      if $S_a^c = \emptyset$ **then**

        $\mathcal{A}^f \leftarrow \mathcal{A}^f \cup \{a\}$ and $\mathcal{A}^u \leftarrow \mathcal{A}^u \setminus \{a\}$

        $\delta^c \leftarrow \delta^c \cup \{(\psi_a, a)\}$

      else

        $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$

        $\delta^w \leftarrow \delta^w \cup \{(\psi_a, a)\}$

        $\mathcal{P}' \leftarrow \mathcal{P}' \cup S_a^c$

    else

      $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$

      $\mathcal{P}' \leftarrow \mathcal{P}' \cup S_a^w$

  $\delta^c \leftarrow \texttt{Simplify}(\delta^c)$ and $\delta \leftarrow \texttt{Simplify}(\delta^c \cup \delta^w)$

  $S^c \leftarrow \texttt{IsValid}((\mathcal{C} \wedge \phi) \leftrightarrow [\mathcal{C} \wedge (\bigvee_{(\psi_a, a) \in \delta} \psi_a)])$

  if $S^c = \emptyset$ **then return** δ ;

  else $\mathcal{P}' \leftarrow (\mathcal{P}' \cup S^c) \setminus \mathcal{P}$ and $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}'$;

(1) s is winning states; and (2) s is not an applicable state for a, or the successor state $\tau(s, a)$ is not a losing state. We also examine the completeness property of (ψ_a, a). This is because, in some cases, action a is the unique action that changes some winning states to a losing state, but ψ_a may not involve these winning states. We therefore enlarge the set $\mathcal{P}$ by the above winning states (Line 17). (When and how does Algorithm 2 generate a new set of actions ??) (Is it possible consider incomplete rule ??)

After obtaining a set δ of winning rules, we will check if δ forms a winning strategy, that is, verifying if the formula $\bigvee_{(\psi_a,a)\in\delta} \psi_a$ is the winning formula (Line 18). Since we have obtained the winning formula ϕ, it suffices to check if ϕ and $\bigvee_{(\psi_a,a)\in\delta} \psi_a$ are logically equivalent under the background theory $\mathcal{C}$. If it is the case (that is, $S^* = \emptyset$), the algorithm yields the simplified winning strategy (Line 20). The simplification procedure eliminates some rules that are covered by the other rules. Given a set of rules (ψ_i, a_i) for $1 \leq i \leq n$, we say (ψ_i, a_i) is redundant, iff $\psi_i \models \bigvee_{j\neq i} \psi_j$. In this case, the rule (ψ_i, a_i) can be eliminated since performing some action a_j in every state satisfying ψ_i leads to a losing state. If the formula $\bigvee_{(\psi_a,a)\in\delta} \psi_a$ is not the winning formula, then some winning states s do not have an semi-ground action a such that the successor state $\tau(s, a)$ is a losing state. The set $\mathcal{P}$ will be extended by S^* for the next iteration of the main loop (Line 22).

Example 7. Table 3 shows the synthesis process of the winning strategy of Take-away-3 game. In the first iteration, four actions $take(1)$, $take(2)$, $take(3)$ and $take(v)$ as well as their corresponding condition ψ_a are generated. The two rules $(v = 2, take(2))$ and $(v = 3, take(3))$ are winning but incomplete. The two states 6 and 7 are the evidences of incompleteness of the above two rules, respectively. The other two rules $(v \equiv_4 1, take(1))$ and $(v \geq 1 \wedge v < 4, take(v))$ are both winning and complete and hence constitute a strategy. However, this strategy is incomplete since the formula $(v \equiv_4 1) \vee (v \geq 1 \wedge v < 4)$ ignores the winning state 9 that is a counterexample provided by the SMT solver.

In the second iteration, a new action $take(v - 4)$ is added into the set $\mathcal{A}_p$.

Round	$\mathcal{A}_p$	$\mathcal{P}_a$	$\mathcal{N}_a$	ψ_a	S'	S^*
1	$take(1)$	$1,5$	$0,2,3,4$	$v \equiv_4 1$	$\emptyset$	9
	$take(2)$	2	$0,1,3,4,5$	$v = 2$	6	
	$take(3)$	3	$0,1,2,4,5$	$v = 3$	7	
	$take(v)$	$1,2,3$	$0,4,5$	$v \geq 1 \wedge v < 4$	$\emptyset$	
2	$take(2)$	$2,6$	$0,1,3,4,5,7,9$	$v \equiv_4 2$	$\emptyset$	$\emptyset$
	$take(3)$	$3,7$	$0,1,2,4,5,6,9$	$v \equiv_4 3$	$\emptyset$	
	$take(v-4)$	$5,6,7$	$0,1,2,3,4,9$	$v \geq 5 \wedge v < 8$	$\emptyset$	

Table 3: A run of synthesizing the winning strategy

The condition of this action together with $take(2)$ and $take(3)$ which fail in the first iteration are generated. The three rules $(v \equiv_4 2, take(2))$, $(v \equiv_4 3, take(3))$ and $(v \geq 5 \wedge v < 8, take(v-4))$ are winning and complete. We put them together and obtain the final winning strategy $\{(v \equiv_4 1, take(1)), (v \equiv_4 2, take(2)), (v \equiv_4 3, take(3)), (v \geq 1 \wedge v < 4, take(v)), (v \geq 5 \wedge v < 8, take(v-4))\}$. □

5.2. Correctness verification

In this subsection, we present the constraints for the winning strategy that is used to verify the winning and completeness property of rules.

Definition 12. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- The constraint for the winning rule (ψ, a) is $(\mathcal{C} \wedge \phi \wedge \psi) \rightarrow \{\texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']\}$.
- The constraint for the complete rule (ψ, a) is $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']] \rightarrow \psi$.

Theorem 7. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. Then, a rule (ψ, a) is winning (resp. complete) iff the constraint for the winning (resp. complete) rule illustrated in Definition 12 is valid.*

Proof. We first verify the correspondence between the winning rule and its constraint.

($\Rightarrow$): Assume that (ψ, a) is a winning rule. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the definition of winning rules (cf. Definition 5), a is applicable in s and $\tau(s, a)$ is a losing state. It follows that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']$. Hence, the constraint for the winning rule is valid.

($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the winning rule is valid. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the constraint for the winning rule, we get that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']$. It follows that a is applicable in s and $\tau(s, a)$ is a losing state. Hence, (ψ, a) is a winning rule.

We then verify the correspondence between the complete rule and its constraint.

($\Rightarrow$): Assume that (ψ, a) is a complete rule. Let s be a state satisfying $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. It follows that $\tau(s, a)$ is a losing state. By the definition of complete rules (cf. Definition 5), $s \models \psi$. Hence, the constraint for the complete rule is valid.

($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the complete rule is valid. Let s be a state s.t. $\tau(s, a)$ is a losing state. It follows that $s \models \exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. By the constraint for the complete rule, we get that $s \models \psi$. Hence, (ψ, a) is a complete rule. □

Each rule generated in Algorithm 4 is guaranteed to be complete and winning (According to the new algorithm ??). In addition, the formula $\bigvee_{(\psi_a, a) \in \delta} \psi_a$ is the winning formula. We therefore obtain soundness theorem of Algorithm 4 for synthesizing winning strategies.

Corollary 3. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The set of rules returned by Algorithm 4 is a complete and winning strategy of Π.*

Proof sketch: We first verify the set δ of rules returned by Algorithm 2 is a winning strategy. Line 13 checks if a rule (ψ_a, a) is winning. If $S' = \emptyset$, then the constraints for the winning rule are valid (Line 14). By Theorem 6, (ψ_a, a) is a winning rule and Line 15 adds it into δ. Hence, δ is a winning strategy.

We then verify the δ is a complete strategy. Line 19 checks if the formula $(\bigvee_{(\psi_a,a)\in\delta}\psi_a)$ is the winning formula. If $S^* = \emptyset$, then the set of rules is a complete strategy (Line 15). □

6. The misère rule

In this section, we apply our approach to learning winning formulas and strategies under the misère rule. The definitions of sets of winning states and losing states are the same as Definition 1 except that all ending states are winning states. In addition, we preclude executing an action in winning but ending states. It is easy to carry forward our approach to computing winning strategies via the following minor modifications. (1) In the step of synthesizing winning formulas, classifying legal states into winning and losing states is based on the new definition. (2) The first and second constraints for winning formulas (cf. Definition 11) are replaced by $\mathcal{C} \wedge \mathcal{E} \rightarrow \phi$ and $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \phi) \rightarrow \exists\mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$, respectively. (3) The constraint for the winning rule (ψ, a) (cf. Definition 7) is replaced by $\mathcal{C} \wedge \neg\mathcal{E} \wedge \psi \rightarrow \{\texttt{pre}(a) \wedge \forall\mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$.

7. Experimental evaluation

This section first presents implementation of our proposed algorithms and the benchmarks we establish in the experiments, and then analyzes the experimental results.

7.1. Implementation

We implemented the synthesis approach for winning formulas and strategies, illustrated in Algorithms 1 and 4 in Python 3.7. The two variants, based on different candidate formula generation methods, are referred to as SynEnum (enumeration-based) and SynMS (exact binary classification-based). Our tool takes as input a PDDL-like file that formalizes an ICG and outputs the winning formula and strategy for this ICG. We employ Z3 [**?**] as the SMT solver to

verify the correctness of the winning formula and strategy as well as to determine if a state is winning (??). We adopt NuWLS [**?**] as the partial MaxSAT solver in order to choose the small set of generated arithmetic atoms. NuWLS is an incomplete partial MaxSAT solver and may return a sub-optimal solution. This however has no influence on the soundness of the chosen set as the solution satisfies the hard constraint. By Theorem 5, we can always construct a formula from the chosen set that distinguishes the set of positive states from that of negative states. The time-out bounds for Z3 and NuWLS are 10 and 3 seconds, respectively. The experiments were conducted on a PC with an Intel Core i7-7700 3.60 GHz CPU and 16GB RAM running under Windows 10, with a runtime cutoff of $1,200$ seconds. Source codes are provided in this link[3].

7.2. Benchmarks

We evaluated the performance of our approach SynEnum and SynMS on a large number of ICG domains that consist of the following 5 categories: Subtraction [**?**], Nim [**?**], Wythoff [**?**], Welter [**?**] and Chomp [**?**]. We have provided a detailed description and formalization of these games in Section Appendix. Each category contains multiple classes of ICGs, including some extensions or variants of the original game. The PDDL file of each test case are given in this link[4]. Since the winning formula and strategy are not LIA-definable or unknown in some classes of ICGs, we impose certain limitations on these classes in order to guarantee that they have LIA-definable solution. For example, the winning formula of 3-rowed Chomp game is unknown. We restrict the maximum number of cookies on the 3rd row to be a fixed integer and obtain a restricted version of 3-rowed Chomp which has an LIA-definable solution. In summary, the benchmark consists of a total of 32 classes with $3,720$ test cases, including the 46 test cases in [**?**]. (add more detailed description of each test cases ??)

[3] XXXX

[4] https://github.com/YMH0607/domain/tree/master

7.3. Experimental Results

We compared the performance of the SynEnum and SynMS under the normal and misère rules, respectively, and provided detailed experimental data tables. In each table, the columns "Solved" denotes the number of test cases that solved by each approach, and the figure in parentheses denotes the number of test cases that solved by the corresponding approach but not by the other approach. The columns "Time" and "Size" denote the average time and size of solved test cases for each approach, and the figure in parentheses denotes the average time and size of the test cases solved by both approaches, respectively. The optimal data of each group are in bold.

7.3.1. Experimental Results under normal rule

Comparison on Sub game. We summarize the results of two approaches about synthesizing winning formulas in the Subtraction game, as shown in Table 4. Out of 479 test cases, SynEnum successfully solved 124 test cases, while SynMS solved 363 test cases, achieving a synthesis success rate of 75.7%, which is approximately 2.93× higher than SynEnum's. Notably, we found that for each category of game, SynMS is able to solve all the test cases that SynEnum can solved. For the commonly solved 124 test cases, we can see that the running time of SynEnum is 159.13 seconds, compared to just 6.99 seconds of SynMS, resulting in a significant reduction.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	11.2(11.20)	**8.87**(**8.87**)	**6.97**(**6.97**)	8.93(8.93)
Subtraction	385	86(0)	**315**(229)	201.89(201.89)	**63.73**(**6.68**)	**10.92**(**10.92**)	64.93(25.71)
T,d-I-Mark-Game	64	8(0)	**18**(10)	254.1(254.10)	**100.06**(**3.28**)	**18.88**(**18.88**)	76.22(71.75)
Total	479	124(0)	**363**(239)	159.13(159.13)	**61.0**(**6.99**)	**10.48**(**10.48**)	60.86(24.62)

Table 4: Results of synthesizing winning formulas for Sub games under the normal rule

For a deeper analysis of runtime behavior, we show the number of solved test cases after a certain amount of time in Figure 2(a). In only 10 seconds,

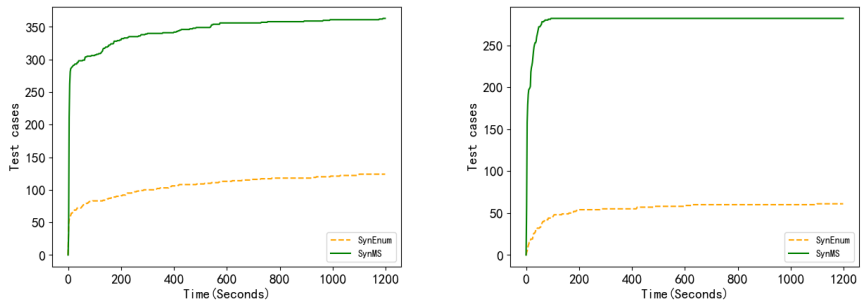
Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	14(0)	**30**(16)	58.54(58.54)	**16.86**(**4.51**)	**91.43**(**91.43**)	236.13(106.07)
Subtraction	385	45(1)	**247**(203)	121.33(99.11)	**10.71**(**2.33**)	**46.58**(**46.14**)	306.86(117.84)
T,d-I-Mark-Game	64	2(0)	**5**(3)	206.96(206.96)	**28.50**(**18.64**)	**64.0**(**64.00**)	130.4(79.00)
Total	479	61(1)	**282**(222)	109.73(93.24)	**11.68**(**3.38**)	**57.44**(**57.30**)	296.21(113.80)

Table 5: Results of synthesizing winning strategies for Sub games under the normal rule

SynMS can solve approximately 284 test cases. In comparison, SynEnum only completes 124 test cases within the timeout $1,200$ seconds. As for the size, SynMS generates formulas larger than SynEnum, with an average size 24.62 on commonly solved test cases, while SynEnum has an average size 10.48. In theory, Enum is guaranteed to find the smallest-size formula via exhaust enumeration[**?**]. SynMS constructs the winning formula in disjunctive normal form based on the selected atoms. In addition, any formula is exponentially smaller than its equivalent one in disjunctive normal form. It is noticeable that SynMS achieves a better performance in computing winning formulas compared to SynEnum although it sacrifices solution size.

Table 5 summarizes the results about synthesizing winning strategies. SynEnum successfully solved 61 test cases, with a success rate of 49.1% and an average running time of 109.73 seconds, while SynMS successfully solved 282 test cases, with a success rate of 77.6% and an average running time of 11.68 seconds. Whether it is the running time for commonly solved test cases or the running time for all test cases, SynMS outperforms SynEnum. This is also evident in Figure 2(b), where SynMS is able to solve 278 test cases in 60 seconds, while SynEnum can only solve 36 test cases. It is worth noting that SynEnum successfully solves 1 test case that SynMS cannot. The winning formulas of the test case is an arithmetic literal (that is, an arithmetic atom or its negation) of large size. SynEnum is more effective than SynMS at solving the test case since it can enumerate formulas up to size 25. SynMS takes some time to choose arithmetic atoms and combine the chosen atoms into an arithmetic formula and

hence generates arithmetic atoms of at most size 7. For the commonly solved 60 test cases, the comparison of formula size is consistent with the performance observed when synthesizing winning formulas.



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 2: Comparison of the running time of synthesizing winning formula and strategy for Sub game under normal rule

Comparison on Nim game. Table 6 presents the results of two approaches about synthesizing winning formulas in the Nim game. The findings reveal that the number of test cases successfully solved by SynEnum and SynMS is comparable. Specifically, SynEnum successfully solved 66 test cases with an average running time of 138.21 seconds, while SynMS solved 73 test cases, but with a faster average running time of 51.82 seconds. The most notable difference between the two approaches is observed in the l-Slow Nim game. Furthermore, we found that there are 29 test cases that SynEnum can solve, but SynMS cannot, a pattern similar to what was observed in the Subtraction game. For the 37 test cases that are commonly solved, SynEnum's average running time is 62.96 seconds, while SynMS's average running time is 23.94 seconds, making SynMS approximately 2.62× faster than SynEnum. However, we found that in certain categories of games, SynEnum performs faster than SynMS on the commonly solved test cases. For example, in the Monotonic Nim game, SynEnum's average running time is 2.29 seconds, while SynMS takes 18.43 seconds. This is due to the simplicity of these test cases. Although the average running time of SynMS

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	1(0)	**2**(1)	**0.51**(**0.51**)	80.84(3.21)	**4.00**(**4.00**)	59.5(6.00)
Monotonic Nim	7	**7**(0)	**7**(0)	**2.29**(**2.29**)	18.43(18.43)	**8.29**(**8.29**)	12.57(12.57)
Empty and Transfer	7	1(0)	**2**(1)	100.92(100.92)	**36.41**(**15.91**)	**12.0**(**12.00**)	26.5(43.00)
Empty All But One	6	2(0)	**3**(1)	269.48(269.48)	**26.17**(**21.04**)	**17.50**(**17.50**)	73.0(45.00)
Empty and Redistribute	6	0(0)	**2**(2)	-(-)	107.91(-)	-(-)	97.50(-)
Entropy Reduction Game	7	0(0)	**1**(1)	-(-)	40.33(-)	-(-)	68.00(-)
Min Game	6	1(0)	**2**(1)	**3.78**(**3.78**)	5.73(7.44)	**10.00**(**10.00**)	67.5(16.00)
l-Slow Nim	180	**35**(27)	14(6)	204.32(56.46)	**38.32**(**42.42**)	**11.60**(**10.25**)	103.21(91.25)
Sharing Nim	7	1(0)	**6**(5)	**0.07**(**0.07**)	83.27(0.73)	**1.00**(**1.00**)	34.83(**1.00**)
Small Nim	7	2(0)	**6**(4)	597.29(597.29)	**24.07**(**6.93**)	**16.50**(**16.50**)	49.33(62.00)
Greedy Nim	7	1(0)	**6**(5)	**0.55**(**0.55**)	159.66(2.98)	**4.00**(**4.00**)	285.17(6.00)
Odd-or-Even Nim	7	0(0)	**6**(6)	-(-)	47.21(-)	-(-)	415.50(-)
Modular Nim	5	**1**(0)	**1**(0)	**1.49**(**1.49**)	2.36(2.36)	**8.00**(**8.00**)	25.0(25.00)
Odd-or-Even Nim	7	0(0)	**6**(6)	-(-)	47.21(-)	-(-)	415.50(-)
Pointed Nim	7	0(0)	**1**(1)	-(-)	221.87(-)	-(-)	44.00(-)
End Nim	7	1(0)	**2**(1)	**5.54**(5.54)	20.48(**1.61**)	**13.00**(**13.00**)	157.5(**13.00**)
Loop End Nim	7	1(0)	**2**(1)	**5.42**(5.42)	32.90(7.04)	**13.00**(**13.00**)	76.5(**13.00**)
Staircase Nim	7	**7**(0)	**7**(0)	**1.09**(**1.09**)	42.98(42.98)	**4.00**(**4.00**)	9.0(9.00)
Circular Nim	15	**5**(2)	3(0)	19.05(**0.66**)	**6.52**(6.52)	**7.40**(**4.67**)	12.0(12.00)
Total	302	66(29)	**73**(36)	138.21(62.95)	**51.82**(**23.94**)	**10.03**(**8.51**)	105.03(33.92)

Table 6: Results of synthesizing winning formulas for Nim games under normal rule

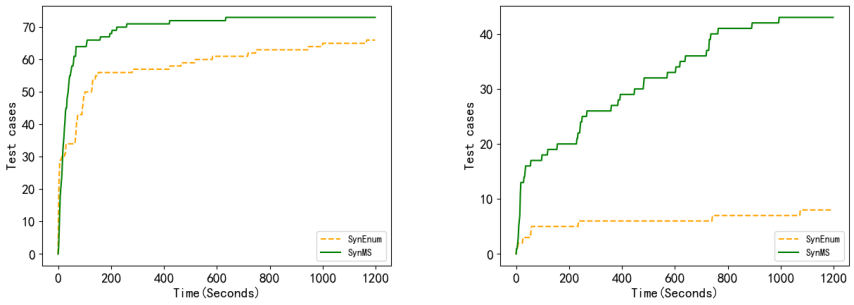
is generally slightly longer than that of SynEnum for these test cases, the time difference between the two approaches typically stays within 50 seconds. Further analysis in Figure 3(a) shows that both approaches are able to solve 75% of the test cases they can solve within 100 seconds. Regarding the size of the formulas, similar to the case in Subtraction game, the formulas generated by SynMS are still larger than those generated by SynEnum.

Table 7 presents the results about synthesizing winning strategies. We can observe that the comparison between SynEnum and SynMS, in terms of the number of solved test cases and average running time, is similar, just as it is in the synthesis of winning formulas. The only difference is that, among

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	0(0)	**1**(1)	-(-)	13.68(-)	-(-)	44.00(-)
Monotonic Nim	7	0(0)	**1**(1)	-(-)	15.87(-)	-(-)	22.00(-)
Empty and Transfer	7	0(0)	**1**(1)	-(-)	15.73(-)	-(-)	284.00(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	**7.77**(**7.77**)	16.71(28.14)	**84.00**(**84.00**)	294.5(154.00)
l-Slow Nim	180	1(0)	**7**(6)	**234.69**(234.69)	384.28(**34.92**)	**108.00**(**108.00**)	358.57(135.00)
Sharing Nim	7	1(0)	**5**(4)	**1.11**(1.11)	79.92(**0.89**)	**1.00**(**1.00**)	185.00(**1.00**)
Small Nim	7	0(0)	**3**(3)	-(-)	353.01(-)	-(-)	463.33(-)
Greedy Nim	7	0(0)	**3**(3)	-(-)	327.74(-)	-(-)	526.00(-)
Odd-or-Even Nim	7	0(0)	**5**(5)	-(-)	306.66(-)	-(-)	1356.60(-)
Modular Nim	5	0(0)	**1**(1)	-(-)	7.34(-)	-(-)	246.00(-)
Pointed Nim	7	0(0)	**1**(1)	-(-)	602.37(-)	-(-)	365.00(-)
End Nim	7	1(0)	**2**(1)	**56.36**(56.36)	501.19(**10.22**)	**149.00**(149.00)	608.00(**74.00**)
Loop End Nim	7	1(0)	**2**(1)	**55.73**(55.73)	450.56(**9.81**)	**149.00**(149.00)	451.00(**74.00**)
Staircase Nim	7	2(0)	**7**(5)	**383.21**(383.21)	479.87(**119.13**)	**155.50**(**155.50**)	399.71(230.00)
Circular Nim	15	1(0)	**2**(1)	1072.6(1072.60)	**25.09**(**32.22**)	126.0(126.00)	**29.00**(**14.00**)
Total	302	8(0)	**43**(35)	**274.33**(274.33)	294.57(**44.31**)	**116.00**(116.00)	458.37(**114.00**)

Table 7: Results of synthesizing winning strategies for Nim games under normal rule

the 8 commonly solved test cases, SynMS produces a smaller strategy size in 3 of them compared to SynEnum. The main reason for this phenomenon is that SynEnum is able to generate a conditional formula ψ_1 containing larger arithmetic atomic formulas for a specific action, which SynMS cannot generate. However, SynMS can generate conditional formulas for other actions thereby indirectly covering all winning states that satisfy the conditional formula ψ_1. Specifically, suppose SynEnum can synthesize the conditional formula ψ_1 for action a_1, where ψ_1 contains larger arithmetic atomic formulas. However, SynMS can synthesize the conditional formula ψ_2 for action a_2, and for any state s, if $s \models \texttt{pre}(a_1) \wedge \forall \mathcal{X}'[\mathcal{T}(a_1) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$ holds, then it must also be true that $s \models \texttt{pre}(a_2) \wedge \forall \mathcal{X}'[\mathcal{T}(a_2) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$ holds. Since SynEnum can also synthe-



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 3: Comparison of the running time of synthesizing winning formula and strategy for Nim game under normal rule

size the corresponding conditional formulas for action a_2, SynMS typically ends up with a smaller strategy size than SynEnum.

Comparison on Welter game. We summarize the results of two approaches about synthesizing winning formulas in the Welter game, as shown in Table 8. SynEnum successfully solved 3 test cases, while SynMS solved 4 test cases, including 2 commonly solved test cases and 2 test cases that SynEnum can not solve but SynMS can. In terms of average running time on the commonly sloved test cases, the difference between the two approaches is very small and can almost be ignored. As shown in Figure 4(a), the trends in solving test cases

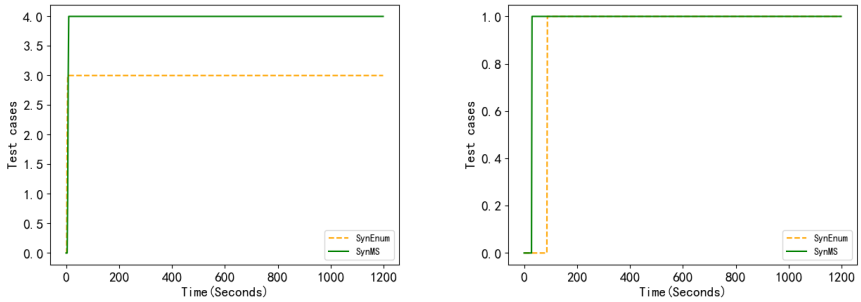
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	**1**(1)	-(-)	8.04(-)	-(-)	73.00(-)
I_welter Game	5	**3**(1)	2(0)	**3.75**(**3.71**)	6.13(6.13)	**8.67**(**8.50**)	88.5(88.50)
Max_welter Game	1	0(0)	**1**(1)	-(-)	7.83(-)	-(-)	73.00(-)
total	7	3(1)	**4**(2)	**3.75**(**3.71**)	7.03(6.13)	**8.67**(**8.50**)	80.75(88.50)

Table 8: Results of synthesizing winning formulas for Welter games under normal rule

over time are very similar for both approaches, with both stabilizing at nearly the same point. Regarding the average formula size for the commonly solved

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
I_welter Game	5	**1**(0)	**1**(0)	87.66(87.66)	**29.22**(**29.22**)	**72.0**(**72.00**)	107.0(107.00)
Max_welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
total	7	**1**(0)	**1**(0)	87.66(87.66)	**29.22**(**29.22**)	**72.00**(**72.00**)	107.00(107.00)

Table 9: Results of synthesizing winning strategies for Welter games under normal rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 4: Comparison of the running time of synthesizing winning formula and strategy for Welter game under normal rule

test cases, the formula size of SynEnum is still smaller than that of SynMS. In terms of strategy synthesis, both SynEnum and SynMS solved only one test case, with their performance in average running time and average strategy size being similar to that of winning formula synthesis.

Comparison on Wythoff game. Table 10 presents the results about synthesizing winning formulas in the Wythoff game. The results show that both SynEnum and SynMS perform excellently in the Wythoff game, successfully solving over $1,700$ test cases, with SynMS solving 582 more test cases than SynEnum. In addition, SynMS is able to solve 647 test cases that SynEnum can not solve. For the commonly solved 1640 test cases, the average running time of SynMS is 8.34 seconds, while SynEnum's is 50.47 seconds, about $6\times$ longer

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	10(0)	**17**(7)	124.9(124.9)	**106.53**(**5.39**)	**10.10**(**10.01**)	89.82(28.10)
l_Wythoff_Game	140	62(6)	**89**(33)	85.19(89.51)	**50.31**(**26.34**)	**10.6**(**10.84**)	127.35(67.12)
Monotonic_l_Diet_Game	105	76(17)	**78**(19)	54.46(51.28)	8.**36**(**6.87**)	**9.62**(**9.41**)	128.29(29.25)
Wythoff with subtraction set S	315	153(1)	**258**(106)	42.89(40.97)	**36.04**(**5.75**)	8.05(7.99)	70.98(26.86)
Wythoff with a function f	168	95(7)	**125**(37)	58.08(49.80)	**41.20**(**9.18**)	**8.65**(**8.70**)	97.09(55.82)
1st(l_1,l_2)restricted Wythoff	315	162(0)	**251**(89)	53.1(53.10)	**33.97**(**5.07**)	**6.84**(**6.84**)	77.96(20.09)
2nd(l_1,l_2)restricted Wythoff	600	503(1)	**575**(73)	21.8(21.36)	**13.22**(**4.92**)	**7.80**(**7.78**)	41.48(22.31)
3rd(l_1,l_2)restricted Wythoff	700	296(33)	**408**(145)	136.89(118.87)	**24.24**(**19.68**)	**10.71**(**11.12**)	119.59(72.56)
4th(l_1,l_2)restricted Wythoff	560	348(0)	**486**(138)	35.31(35.31)	**17.84**(**4.59**)	**6.9**(**6.85**)	57.45(15.89)
Total	2924	1705(65)	**2287**(647)	55.79(50.47)	**24.52**(**8.34**)	**8.34**(**8.29**)	75.84(33.82)

Table 10: Results of synthesizing winning formulas for Wythoff games under normal rule

than SynMS. Despite the time difference, it is worth noting that for each game category, the average running time for both approaches stays within a relatively reasonable range on the commonly solved test case, under 125 seconds. As for the formula size, for each game category, SynEnum typically synthesizes smaller formulas, usually around 15. In contrast, SynMS tends to synthesize larger formulas, once again reflecting the trade-off between the size of the solutions and the running time during the synthesis of the winning formulas. Further analysis of Figure 5 (a) shows that both approaches can solve over 1200 test cases within only 5 seconds, with these test cases being relatively simple. The distinction between the two approaches becomes evident after 5 seconds: SynEnum's solving speed gradually slows down over time, while SynMS maintains a consistently fast solving speed.

The results about synthesizing winning strategies are shown in the Table 11 and Figures 5(b). It is evident that SynEnum significantly lags behind SynMS in terms of synthesizing winning strategies. SynEnum successfully solved only 477 test cases, with a success rate of less than 30%, while SynMS solved $1,266$ test cases and achieved a success rate of over 50%. This difference arises because the winning strategies for these test cases are complex, which are com-

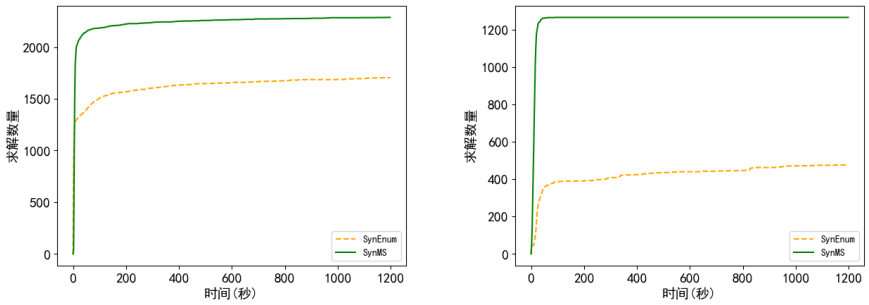
posed of a larger number of arithmetic atoms. Figure 5 (b) further shows that within 50 seconds, SynMS can solve over 1,200 test cases, whereas SynEnum can handle fewer than 400. It is worth mentioning although SynEnum's average running time for each category game is still higher than that of SynMS on the commonly solved test cases, the running time for both approaches did not exceed 160 seconds and remained within a short time range. At the same time, whether in synthesizing winning formulas or winning strategies, the average size of SynEnum is smaller than that of SynMS.

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	2(0)	**8**(6)	27.42(27.42)	**13.36**(**11.21**)	**85.00**(**85.00**)	211.5(41.50)
l_Wythoff_Game	140	1(0)	**51**(50)	20.32(20.32)	**13.90**(**11.07**)	**82.00**(**82.00**)	349.92(**82.00**)
Monotonic_l_Diet_Game	105	24(2)	**71**(49)	63.95(69.72)	**8.75**(**4.97**)	**31.29**(**33.09**)	227.7(46.05)
Wythoff with subtraction set S	315	52(0)	**128**(76)	48.33(48.33)	**11.94**(**11.62**)	**104.08**(**104.08**)	279.27(148.73)
Wythoff with a function f	168	4(0)	**76**(72)	16.93(16.93)	**10.84**(**12.55**)	**82.00**(**82.00**)	287.34(98.75)
1st(l_1,l_2)restricted Wythoff	315	86(0)	**136**(50)	71.14(71.14)	**14.18**(**15.66**)	**117.76**(**117.76**)	277.54(203.40)
2nd(l_1,l_2)restricted Wythoff	600	36(1)	**211**(176)	**0.42**(0.06)	3.2(**0.05**)	**2.69**(**1.00**)	102.41(**1.00**)
3rd(l_1,l_2)restricted Wythoff	700	93(4)	**243**(154)	157.23(134.83)	16.60(8.67)	104.55(100.7)	420.33(125.52)
4th(l_1,l_2)restricted Wythoff	560	179(0)	**342**(163)	194.79(194.79)	11.21(11.62)	127.73(127.73)	282.42(218.22)
Total	2924	477(7)	**1266**(796)	125.4(121.65)	**11.26**(**10.63**)	**103.88**(**103.63**)	277.54(163.97)

Table 11: Results of synthesizing winning strategies for Wythoff games under normal rule

Comparison on Chomp game. We summarize the results of two approaches about synthesizing winning formulas in the Chomp game, as shown in Table 12. Both SynEnum and SynMS are only able to solve 2 test cases. Since the winning formulas for these test cases are relatively simple, SynEnum's average solving time is shorter than that of SynMS on the commonly solved test cases. Additionally, the average size of the winning formulas for these test cases is identical, with both having a size of 6.

Table 13 presents the results about synthesizing winning strategies. Both SynEnum and SynMS are able to solve 1 test case, and the comparison of their average running times and sizes is consistent with those observed in the synthesis



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 5: Comparison of the running time of synthesizing winning formula and strategy for Wythoff game under normal rule

of winning formulas. Figure 6 further confirms this.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**1.78**(**1.78**)	7.26(7.26)	**8.00**(**8.00**)	**8.00**(**8.00**)
L_shape Chomp Game	1	**1**(0)	**1**(0)	**1.09**(**1.09**)	9.1(9.10)	**4.00**(**4.00**)	**4.00**(**4.00**)
Total	8	**2**(0)	**2**(0)	**1.44**(**1.44**)	8.19(8.18)	**6.00**(**6.00**)	**6.00**(**6.00**)

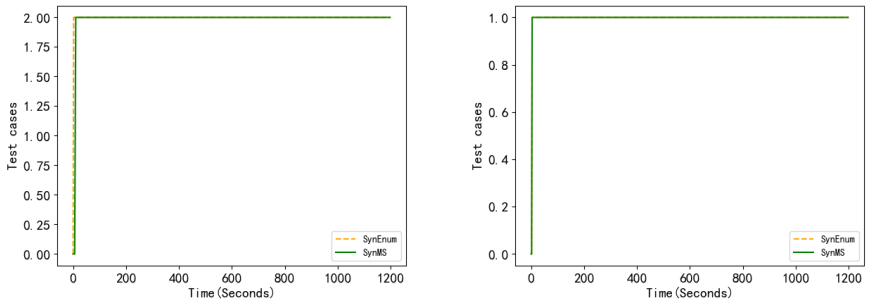
Table 12: Results of synthesizing winning formulas for Chomp games under normal rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**2.30**(**2.30**)	3.50(3.50)	**29.00**(**29.00**)	**29.00**(**29.00**)
L_shape Chomp Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	8	**1**(0)	**1**(0)	**2.30**(**2.30**)	3.50(3.50)	**29.00**(**29.00**)	**29.00**(**29.00**)

Table 13: Results of synthesizing winning strategies for Chomp games under normal rule

7.3.2. Experimental Results under misère rule

Comparison on Sub game. Table 14 presents the results about synthesizing winning formulas under the misère rule. Out of 479 test cases, the



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 6: Comparison of the running time of synthesizing winning formula and strategy for Chomp game under normal rule

number of test cases solved by SynEnum is 144 test cases while SynMS is able to solve 387 test cases including 243 test cases that SynEnum cannot solve, demonstrating a substantial improvement over SynEnum. Compared to the normal rule, SynEnum solves 20 more test cases, and SynMS solves 24 more test cases, with these additional test cases mainly concentrated in the T,d-I-Mark-Game category. This is because, under the misè rule, the winning formulas for these test cases become simplified. Notably, SynMS can solve all the test cases that SynEnum can, reflecting similar performance to the normal rule. For the test cases commonly solved in the Subtraction game, SynMS had an average running time of 14.87 seconds, while SynEnum took 220.31 seconds. Although SynEnum performed worse than SynMS for this group of test cases, it outperformed SynMS on 66 simpler test cases from the Take-away and T,d-I-Mark-Game categories. Further analysis of Figure 8(a) shows that SynEnum and SynMS successfully solved 99 test cases within 20 seconds, including almost all commonly solved test cases from the Take-away and T,d-I-Mark-Game categories. After 20 seconds, the test cases that SynEnum continued to solve mainly involved more complex winning formulas synthesis problems.

Table 15 and Figure 8(b) summarize the results of synthesizing winning strategies. SynEnum successfully solved 89 test cases, including 34 test cases

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	**11.14**(**10.47**)	18.71(18.71)	**6.97**(**6.97**)	8.97(8.97)
Subtraction	385	78(0)	**305**(227)	220.31(220.31)	**98.08**(**14.87**)	**10.90**(**10.90**)	68.82(24.28)
T,d-I-Mark-Game	64	36(0)	**52**(16)	**1.61**(**1.61**)	3.22(3.43)	**6.00**(**6.00**)	8.00(8.00)
Total	479	144(0)	**387**(243)	122.06(122.70)	**79.34**(**12.77**)	**8.85**(**8.87**)	56.13(17.08)

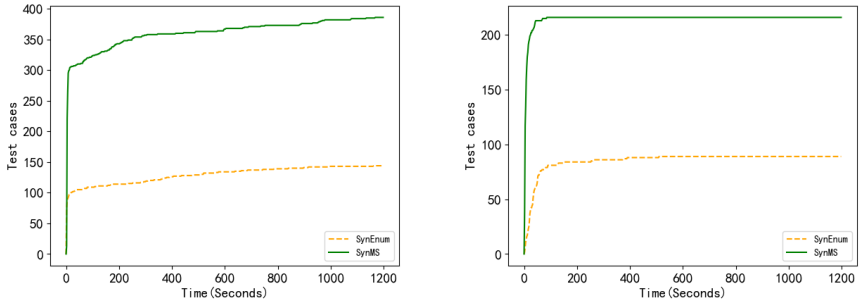
Table 14: Results of synthesizing winning formulas for Sub games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**11**(5)	6(0)	31.37(10.28)	**2.45**(**2.45**)	63.55(**41.33**)	**51.17**(51.17)
Subtraction	385	42(8)	**180**(146)	52.14(58.45)	**8.19**(**2.16**)	**33.14**(**32.56**)	231.16(107.24)
T,d-I-Mark-Game	64	**36**(21)	30(15)	57.24(82.04)	**8.41**(**10.06**)	66.17(**50.07**)	**42.87**(50.20)
Total	479	89(34)	**216**(161)	51.64(59.63)	**8.06**(**4.35**)	**50.26**(**38.29**)	200.0(85.56)

Table 15: Results of synthesizing winning strategies for Sub games under misère rule

with larger arithmetic atoms that SynMS could not solve. For the test cases commonly solved, SynMS typically had a shorter average running time than SynEnum. Regarding strategy size, SynMS typically produced larger strategies compared to SynEnum, which aligns with the results of synthesizing the winning formulas.

Comparison on Nim game. Table 16 and Table 17 presents the results about synthesizing winning formulas and strategies under the misère rule in the Nim game. Compared to the normal rule, the winning formulas of the test cases in the game become more complex. As a result, both SynEnum and SynMS saw a decrease in the number of solved test cases about synthesizing winning formulas. Compared to the normal rule, SynEnum solved 23 test cases, 43 fewer than before, with a decrease in the solve rate of 14.23%. SynMS faced a more severe situation, solving only 17 test cases, 56 fewer than before, with a decrease in the solve rate of 18.54%. An in-depth analysis of Figure 8(a) shows that, although SynMS solved fewer test cases than SynEnum, it is able to synthesize 17



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 7: Comparison of the running time of synthesizing winning formula and strategy for Sub game under misère rule

test cases within the first 28 seconds, while SynEnum could only synthesize 12 test cases. After 28 seconds, SynEnum shows greater advantages in solving test cases with winning formulas consisting of larger arithmetic atoms. Moreover, in the synthesis of winning strategies, SynMS showed a clear advantage in synthesizing complex strategies. SynMS is able to synthesize all 3 test cases that SynEnum successfully solved and also solved 2 test cases that SynEnum can not synthesize. Figure 8(b) clearly shows that SynMS outperforms SynEnum in synthesis efficiency at all times. For the test cases solved by both approaches, the comparison of the average formula and strategy size are similar to those observed in the previous games.

Comparison on Welter game. The results about synthesizing winning formulas and strategies are shown in Table 18, 19, and Figures 9. Similar to the case with Nim, the Welter game becomes more complex under the misère rule, resulting in a decrease in the number of solved test cases for both SynEnum and SynMS. In terms of synthesizing winning formulas, SynEnum solved only 1 test case, while SynMS is able to solve only 3 test cases. For the 1 test case solved by both, the running times of SynEnum and SynMS are almost identical, both within a very short time of 12 seconds. However, in the synthesis of winning strategies, SynEnum is unable to solve any test cases, while SynMS successfully

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	**1**(0)	**1**(0)	422.33(422.33)	**5.35**(**5.35**)	**20.0**(**20.0**)	130.0(130.0)
Monotonic Nim	7	**2**(1)	1(0)	**1.83**(**1.50**)	13.38(13.38)	10.00(9.00)	**8.00**(**8.00**)
Empty and Transfer	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	**2.30**(**2.30**)	7.8(10.42)	**10.00**(**10.00**)	128.50(42.00)
l-Slow Nim	180	**7**(5)	2(0)	37.28(**3.83**)	**6.84**(6.84)	**9.57**(**8.50**)	88.50(88.50)
Sharing Nim	7	**1**(0)	**1**(0)	39.87(39.87)	**26.77**(**26.77**)	**17.00**(**17.00**)	23.00(23.00)
Small Nim	7	**2**(1)	1(0)	495.75(19.77)	**3.25**(**3.25**)	19.00(**18.00**)	**18.00**(**18.00**)
Greedy Nim	7	**1**(0)	**1**(0)	562.30(562.30)	**5.09**(**5.09**)	**20.00**(**20.00**)	130.00(130.00)
Odd-or-Even Nim	7	0(0)	**1**(1)	-(-)	18.30(-)	-(-)	295.00(-)
Modular Nim	5	**1**(0)	**1**(0)	**1.85**(**1.85**)	4.06(4.06)	**8.00**(**8.00**)	25.00(25.00)
Pointed Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
End Nim	7	**1**(0)	**1**(0)	100.50(100.50)	**2.82**(**2.82**)	**18.00**(**18.00**)	**18.00**(**18.00**)
Loop End Nim	7	**1**(0)	**1**(0)	965.87(965.87)	**10.92**(**10.92**)	**20.00**(**20.00**)	79.00(79.00)
Staircase Nim	7	**2**(1)	1(0)	**1.26**(**0.41**)	13.68(13.68)	**5.00**(**4.00**)	6.00(6.00)
Circular Nim	15	**3**(0)	**3**(0)	149.52(149.52)	**8.10**(**8.10**)	**10.00**(**10.00**)	65.00(65.00)
Total	302	**23**(8)	17(2)	165.31(171.53)	**9.25**(**8.92**)	**12.09**(**12.73**)	80.05(56.73)

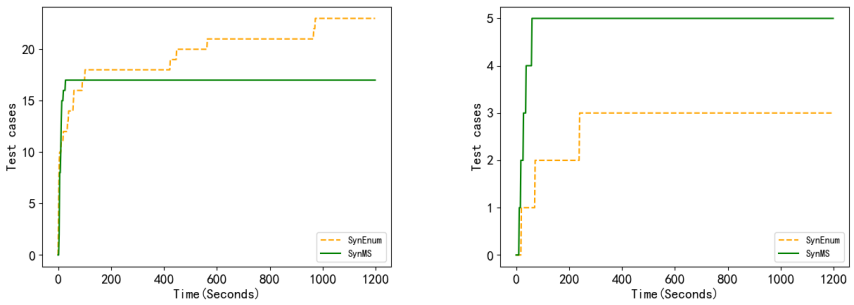
Table 16: Results of synthesizing winning formulas for Nim games under misère rule

solved only 1 test case. This change is also clearly evident in Figure 9.

Comparison on Wythoff game. Table 20 presents the results about synthesizing winning formulas under the misère rule in the Wythoff game. Similar to the cases with Nim and Welter, a similar situation also occurs in the synthesis of winning formulas in the Wythoff game. SynEnum can solve $1,416$ test cases, solving 321 fewer than before. SynMS can solve $2,185$ test cases, solving 102 fewer than before. These test cases are mainly concentrated in games like Monotonic_l_Diet_Game, st1_lm_restricted, and nd2_lm_restricted. Although both SynEnum and SynMS saw a decrease in the number of solved test cases, the number of solved test cases by both remained above $1,400$. Under the normal rule, both SynEnum and SynMS are able to solve over $1,200$ test cases

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Monotonic Nim	7	0(0)	**1**(1)	-(-)	59.72(-)	-(-)	96.0(-)
Empty and Transfer	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	18.70(18.70)	**13.77**(**17.32**)	**39.00**(**39.00**)	232.5(70.00)
l-Slow Nim	180	**1**(0)	**1**(0)	239.23(239.23)	**37.23**(**37.23**)	**120.00**(**120.00**)	173.00(173.00)
Sharing Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Small Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Greedy Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Odd-or-Even Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Modular Nim	5	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Pointed Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
End Nim	7	**1**(0)	**1**(0)	70.36(70.36)	**26.25**(**26.25**)	149.00(149.00)	**74.00**(**74.00**)
Loop End Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Staircase Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Circular Nim	15	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	302	3(0)	**5**(2)	109.43(109.43)	**30.15**(**26.93**)	**102.67**(**102.67**)	161.6(105.67)

Table 17: Results of synthesizing winning strategies for Nim games under misère rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

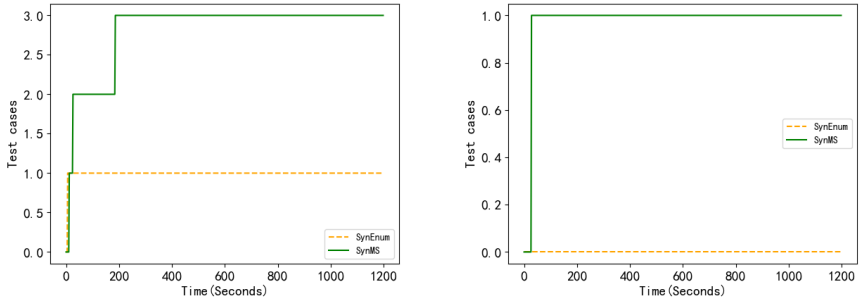
Figure 8: Comparison of the running time of synthesizing winning formula and strategy for Nim game under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	**1**(1)	-(-)	185.35(-)	-(-)	1166.00(-)
I_welter Game	5	**1**(0)	**1**(0)	**5.80**(**5.80**)	11.70(11.70)	**8.00**(**8.00**)	9.00(9.00)
Max_welter Game	1	0(0)	**1**(1)	-(-)	25.26(-)	-(-)	97.00(-)
Total	7	1(0)	**3**(2)	**5.80**(**5.80**)	74.10(11.70)	**8.00**(**8.00**)	424.00(9.00)

Table 18: Results of synthesizing winning formulas for Welter games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
I_welter Game	5	0(0)	**1**(1)	-(-)	27.87(-)	-(-)	115.00(-)
Max_welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	7	0(0)	**1**(1)	-(-)	27.87(-)	-(-)	115.00(-)

Table 19: Results of synthesizing winning strategies for Welter games under misère rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 9: Comparison of the running time of synthesizing winning formula and strategy for Welter game under misère rule

within 5 seconds. Similarly, under the misère rule, they are also able to solve more than 1,000 test cases within 5 seconds as shown in Figure 10(a). Similar to its performance in Nim and Welter games, SynEnum solved 203 fewer test cases under the misère rule compared to the normal rule, primarily in

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	7(0)	**17**(10)	**5.06**(**5.06**)	29.97(6.41)	**10.00**(**10.0**)	155.71(36.86)
l_Wythoff_Game	140	55(0)	**89**(34)	62.45(62.45)	**51.39**(**22.78**)	**11.42**(**11.42**)	117.51(61.35)
Monotonic_l_Diet_Game	105	44(0)	**67**(23)	43.25(43.25)	**25.04**(**4.57**)	**8.89**(**8.89**)	75.25(37.91)
Wythoff with subtraction set S	315	158(0)	**260**(102)	57.89(57.89)	**26.56**(**7.06**)	**10.68**(**10.68**)	142.43(63.65)
Wythoff with a function f	168	82(6)	**120**(44)	99.53(73.69)	**18.72**(**11.14**)	**11.93**(**11.54**)	118.65(60.33)
1st(l_1,l_2)-restricted Wythoff	315	138(1)	**243**(106)	122.17(122.4)	**53.46**(**12.78**)	**10.93**(**10.91**)	154.37(65.24)
2nd(l_1,l_2)-restricted Wythoff	600	365(22)	**494**(151)	73.65(76.78)	**32.04**(**17.28**)	**10.77**(**10.50**)	99.25(61.41)
3rd(l_1,l_2)-restricted Wythoff	700	271(3)	**426**(158)	67.77(61.44)	**41.12**(**12.68**)	**11.61**(**11.51**)	108.24(57.09)
4th(l_1,l_2)-restricted Wythoff	560	296(1)	**469**(174)	75.04(74.62)	**34.37**(**6.17**)	**11.37**(**11.37**)	125.18(67.67)
Total	2924	1416(33)	**2185**(802)	75.57(73.54)	**35.87**(**11.83**)	**11.09**(**10.98**)	119.35(61.61)

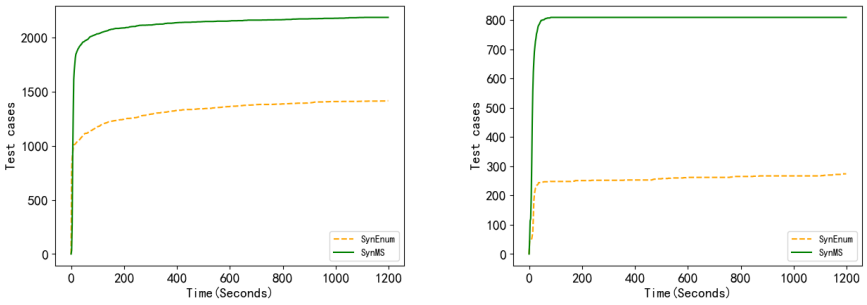
Table 20: Results of synthesizing winning formulas for Wythoff games under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	1(0)	**6**(5)	17.80(17.80)	**17.27**(**12.15**)	**66.00**(**66.00**)	378.83(50.00)
l_Wythoff_Game	140	10(0)	**44**(34)	18.07(18.07)	**15.91**(**7.71**)	**62.80**(**62.80**)	454.02(81.00)
Monotonic_l_Diet_Game	105	29(0)	**61**(32)	79.64(79.64)	**10.74**(**9.08**)	**48.66**(**48.66**)	172.59(92.86)
Wythoff with subtraction set S	315	19(1)	**81**(63)	178.89(159.71)	**15.30**(**10.72**)	**80.11**(**76.22**)	402.47(178.17)
Wythoff with a function f	168	12(0)	**54**(42)	16.85(16.85)	**12.51**(**8.78**)	**63.33**(**63.33**)	359.56(75.83)
1st(l_1,l_2)-restricted Wythoff	315	18(1)	**55**(38)	167.62(176.09)	**13.88**(**10.53**)	**75.94**(**76.53**)	355.47(141.59)
2nd(l_1,l_2)-restricted Wythoff	600	94(0)	**112**(18)	103.05(103.05)	**2.09**(**1.98**)	**40.96**(**40.96**)	158.26(138.96)
3rd(l_1,l_2)-restricted Wythoff	700	50(1)	**216**(167)	**17.23**(17.21)	20.70(**8.75**)	**61.24**(**61.29**)	497.74(79.69)
4th(l_1,l_2)-restricted Wythoff	560	41(0)	**180**(139)	44.25(44.25)	**9.83**(**10.50**)	**62.12**(**62.12**)	488.89(71.80)
Total	2924	274(3)	**809**(538)	78.42(77.20)	**13.12**(**6.92**)	**55.52**(**55.12**)	392.56(110.66)

Table 21: Results of synthesizing winning strategies for Wythoff games under misère rule

the rd2_lm_restricted, rd3_lm_restricted, and th4_lm_restricted games. In these games, these test cases are more difficult for SynEnum to solve under the misère rule. Additionally, its performance in terms of runtime, average formula size, and strategy size is similar to that under the normal rule.

Comparison on Chomp game. Table 22 and 23 presents the results of synthesizing winning formulas under the misère rule in the Wythoff game. Both



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 10: Comparison of the running time of synthesizing winning formula and strategy for Wythoff game under misère rule

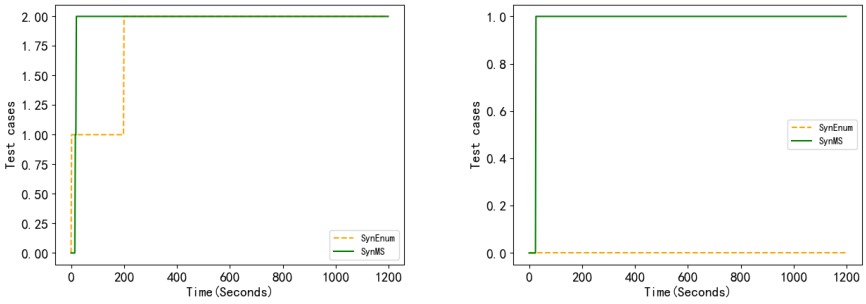
SynEnum and SynMS are only able to solve 2 test cases. For these 2 test cases commonly solved, the average running time for SynEnum is 99.86, while the average running time for SynMS is 16.48, showing a significant reduction in time. As for the size of formulas, the average size of SynMS's formulas is still larger than that of SynEnum. In terms of synthesizing winning strategies, as the game's winning strategies gradually become more complex, SynEnum is unable to solve any test cases, while SynMS was only able to solve 1 test case. Figure 11 illustrates this change.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**0.99**(**0.99**)	14.42(14.42)	**6.00**(**6.00**)	8.00(8.00)
L_shape Chomp Game	1	**1**(0)	**1**(0)	198.73(198.73)	**18.54**(**18.54**)	**16.00**(**16.00**)	101.00(101.00)
total	8	**2**(0)	**2**(0)	99.86(99.86)	**16.48**(**16.48**)	**11.00**(**11.00**)	54.50(54.50)

Table 22: Results of synthesizing winning formulas for Chomp games under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	0(0)	**1**(1)	-(-)	25.68(-)	-(-)	126.00(-)
L_shape Chomp Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
total	8	0(0)	**1**(1)	-(-)	25.68(-)	-(-)	126.00(-)

Table 23: Results of synthesizing winning strategies for Chomp games under misère rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 11: Comparison of the running time of synthesizing winning formula and strategy for Chomp game under misère rule

8. Related work

8.1. Analysis of ICGs

?] introduced an LIA-definable ICG, namely Mark game, and showed that its set of winning states is not ultimately periodic. It follows Theorem 2 that the set of winning states is not LIA-definable. Mark game is a variant of Take-away game and shares the same constraint $\mathcal{C}$ and ending condition $\mathcal{E}$. In this game, the player can take either 1 or $\lfloor \frac{v}{2} \rfloor$ chips from the table, where v denotes the number of remaining chips. The precondition and effect of action $take(k)$ in Mark game are as follows.

- $\texttt{pre}(take(k)) : v \geq 0 \wedge k \geq 1 \wedge (k = 1 \vee v = 2k \vee v = 2k + 1)$;
- $\texttt{eff}(take(k)) : \{\top, v, v - k\}$.

Subtraction game [**?**] is a generalization of the Take-away game. **?**] proved that any Subtraction game admits an LIA-definable set of winning states. Let S be a finite set of positive integers. In this game, each player can take k chips from the pile where $k \in S$. The precondition and effect of action $take(k)$ in Subtraction game are as follows.

- $\texttt{pre}(take(k))$: $(\bigvee_{s \in S} k = s) \wedge v \geq \min(S)$ where $\min(S)$ denotes the smallest integer of S;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$;

?] proved that the problem of deciding whether two LIA-definable ICGs have the same set of winning states is undecidable. **?**] showed that the problem of deciding whether the set of winning states for an LIA-definable ICG is 2-dimensional periodic is undecidable.

8.2. Automatic methods to two player games

Two classical methods for solving the winning strategy of finite-state ICGs are backward induction [**?**] and alpha-beta pruning [**?** **?**]. Backward induction reasons backward from the ending state to the given initial state. For games with large state spaces, the computational complexity of backward induction increases rapidly, leading to low algorithmic efficiency and making it difficult to handle complex games. Alpha-beta pruning is a well-known pruning method for minimax trees. It reduces the number of nodes in the search tree that captures the plays from the given initial state by two players. However, the effectiveness of the alpha-beta pruning method depends on the traversal order of the nodes. If the traversal order is suboptimal, the effectiveness of pruning will be impacted. Recently, **?**] proposed a novel approach to prune the search tree based on the node values calculated by the Sprague-Grundy function under the normal rule. The approach is inspired by the alpha-beta pruning technique, with the key difference being that they use the mex function to assign values to the tree nodes, rather than the traditional minimization/maximization approach. But

the above approaches do not work for infinite-state ICGs. (Add papers about computing SG-values ??)

?] extended the situation calculus for formalizing ICGs. Situation Calculus is a formal logical language used to describe and reason about dynamic systems, particularly suitable for expressing reasoning problems related to actions and changes. They proposed a verification approach for the correctness of winning strategies that are defined in finite state automaton (FAS). The key idea of this approach is to find an invariant for the FSA strategy, which is similar to the concept of a winning formula. It is important to note that their approach only verifies whether a given FSA is a winning strategy, but it does not address the strategy synthesis problem.

Later, they [**?**] developed a CEGIS approach for synthesizing invariant strategies. An invariant strategy is of the form $\phi?;\pi a.a;\varphi?$, meaning whenever ϕ holds, the player can execute an action to enforce φ, and whenever φ holds, executing any action makes φ true regardless of what the action chooses. In fact, the formulas ϕ and φ are the winning and the losing formulas, respectively. Strictly speaking, **?**]'s approach only constructs winning and losing formulas but not winning strategies that specifies which action to do in different partitions of winning states (cf. Definition 5). In addition, as reported in [**?**], their approach is much less efficient than Enum. Hence, we do not compare with their approach in experimental evaluation.

8.3. LTL synthesis

?] first proposed a well-known class of games, namely *linear temporal logic (LTL) games*, where the game is represented by an automaton, and the goal is expressed by a propositional LTL formula. The objective of LTL synthesis is to synthesize a strategy that ensures goal achievement. These games are represented by an automaton, and the goals are expressed by an LTL formula. An LTL formula consists of a set of atomic propositions, logical operators, and temporal operators. The method proposed in [**?**] is an automaton-based approach causing a doubly exponential time complexity. To lower the complexity, **?**] re-

strict the goal to be some fragments of LTL formulas so as to obtain polynomial synthesis approaches, *e.g.*, the safety and reachability games whose goal is of the form $\Box\phi$ and $\Diamond\phi$ respectively where ϕ is propositional. The LTL synthesis problem has been addressed in numerous literature, such as [**?** **?** **?** **?** **?** **?**]. The above methods only focus on games over finite states.

In general, it is undecidable for the synthesis problem of infinite-state games [**?**]. Some decidable restricted classes of infinite-state games are identified, *e.g.*, games on automatic graphs [**?**], pushdown graphs [**?** **?**], and prefix-recognizable graphs [**?**].

Besides automatons, arithmetic formulas are an alternative to formalization for games. Compared to the former, the latter are easier to understand, and hence being more suitable for human beings to give a complete description. Recently, arithmetic formulas are used to represent infinite-state games [**?** **?** **?** **?**]. Our representative framework is also based on arithmetic formulas, and close to their work.

?] devise a sound and relatively complete proof rules for computing winning strategies for safety, reachability and LTL games over infinite states. All impartial combinatorial games considered in this paper are a reachability game over infinite states. The rules for reachability games rely on winning formulas and well-founded relations that are used to guarantee the correctness and termination of the winning strategy respectively. The generation of winning formulas and well-founded relations is solved by the EHSF solver [**?**]. Our approach finds the winning formula via an enumerative approach, and does not generate the well-founded relation as an impartial combinatorial game always ends in a finite number of moves. One major drawback of **?** 's approach is that it requires users to provide hints in the form of templates to synthesize winning strategies, and hence it is not a full mechanization. By contrast, our approach succeeds in computing winning strategies for all games considered in this paper in a fully automated way.

?] propose a fully automated method, called SimSynth, for synthesizing winning strategies for reachability games over infinite states. SimSynth firstly

unrolls games as a bounded variation. Then it reduces the problem of synthesizing the winning strategy for the bounded variation to the linear arithmetic satisfiability problem [**?**]. Finally, it attempts to generalize the strategy so as to hold for all states. The insight of our approach is different from SimSynth. Instead, we first construct the winning formula, and then generate the winning strategy from the cover of the winning formula. SimSynth does not involve the winning formula, which precisely describes the set of winning states. In practice, SimSynth is only able to synthesizes the winning strategy of an ICG instance, and does not scale up well. For example, it fails to solve the 2-piled Nim game for the state $v_1 = 6 \wedge v_2 = 6$ within 10 minutes. By contrast, our approach succeeds in computing generalized winning strategies, which works for not only one state but also all winning states, for the 2-piled Nim game in 3.91 seconds.

?] presents a causality-based algorithm for solving two-player reachability games defined by logical constraints. The approach leverages Craig interpolation to identify necessary subgoals—critical game slices the reachability player must traverse—and recursively decomposes the game along these subgoals. This enables the extraction of structured winning strategies. Recently, **?**] consider introduces a special class of logical reachability games, called GMTs (logical reachability games modulo theories), where the safety player has a limited number of moves per configuration, while the reachability player has no such restriction. The authors show that GMTs can model real-world problems, such as the Cinderella-Stepmother game, and demonstrate how determining the winner in these games can be reduced to solving a system of constrained Horn clauses (CHCs). As shown in [**?**], the running times are significantly lower compared to implementations of **?**].

But these approaches are not suitable for ICGs. In addition to the reasons mentioned above, there are more reason as follows: Firstly, these approaches only generates a winning strategy for a specific initial state, and the generated strategy cannot be generalized to all legal states. Additionally, the formalization of these approaches are based on linear real arithmetic (LRA) rather than LIA. The differences between LIA and LRA are the following: (1) variables in LIA

have an integer domain rather than real one; and (2) LIA contains the congruence modulo relation while LRA does not. The majority of ICGs contain only integer variables and require congruence modulo relation in their winning formula.

Previous research has used LIA-based formalization, but it only supports safety games [**?**]. **?**] proposes a machine learning framework for synthesizing reactive controllers in safety games, particularly those defined over infinite or prohibitively large graphs. The core technique involves a feedback loop between a symbolic teacher component and a learning algorithm, where the learning algorithm learns a winning set, an approximation of the vertices from which the system player can win. The learning process iteratively refines this set using counterexamples provided by the teacher. The paper also introduces a decision tree learning algorithm tailored for this setting, ensuring the efficient learning of winning sets. Recently, **?**] propose a symbolic method that supports LIA for solving infinite-state games, including objectives such as reachability, safety, Büchi, and co-Büchi. This approach introduces acceleration lemmas, a class of simple inductive statements, and leverages them to accelerate the convergence of the game-solving process. However, the two approaches can only compute an approximate winning set starting from a given state, meaning the winning set does not fully cover all possible winning states, and the corresponding winning strategy does not cover all winning paths.

8.4. Generalized planning

A related problem is generalized planning that aims to construct a generalized planning that works for possibly infinitely many planning instances sharing the similar structure [**?**]. **?**] implemented an iterative planner called KPLANNER. The core idea is to first search for planning solutions with no loop structure and variable values less than a threshold, and then identify the common structure in the planning solution based on the built-in pattern matching mechanism of the Prolog language, and extract the planning program with loop structure. On this basis, **?**] proposed a new planner FSAPLANNER. Compared with KPLANNER, FSAPLANNER proposed a more expressive planning

solution form: finite-state automata (FSA). At the same time, they proposed that the existence of planning solutions for one-dimensional planning problems is decidable. Later, **?**] analyzed the complexity of the problem and proved that it is complete in exponential space. However, the above methods are mainly aimed at single-variable scenarios and are difficult to directly extend to complex planning problems with multiple variables.

To solve the multivariate general planning problem, **?**] proposed a method to generate a FSA plan based on state abstraction using 3-valued logic. The method can be applied to loop structures, but the generated solution is not guaranteed to achieve the goal. To solve this problem, they designed an algorithm to calculate the conditions for the planning to terminate and achieve the goal when it only contains simple loop structures, namely applicability condition[**?**]. Nevertheless, the planners synthesized by their method are only allowed to have simple loop structures and cannot synthesize planners with nested loops.

Compiling a generalized planning problem into another planning problem and then indirectly solving the original one is also an effective method. **?**] proposed a method to automatically construct a finite state controller (FSC). This method transforms a partially observable planning problem into a consistent planning problem, and then uses a classical planning solver to indirectly solve the consistent planning problem. However, it is not clear which variants of the original problem the generated FSC can solve. **?**] clarify which original problems the FSC generated by Bonet et al. [**?**] can solve by characterizing the general structure that supports policy generalization. Segovia et al. codify a finite number of generalized planning problems with the same problem domain into a single classical planning problem [**?** **?** **?**]. However, their approach is only applicable to classical planning problems. **?**] proposed a new type of planning problem: Qualitative Numeric Planning (QNP). QNP imposes two restrictions on Numerical Planning: (1) Any formula is a Boolean combination of simple numeric literals in the form of $v > 0$ or $v = 0$; (2) The value of the variable decreases or increases by an uncertain amount. Under these two constraints, the state space of the QNP can be compressed to a finite space of size $2^{|V|}$,

where $|V|$ is the number of numerical variables. Therefore, the QNP problem is decidable. But the expressive power of the QNP proposed above is relatively limited. **?**] used SAT solvers and description logic to learn an abstract QNP from a finite number of problem instances. However, the learned QNP may not be a valid abstraction for the entire set of original problem instances. Later, **?**] studied the necessary conditions to ensure that an abstraction is valid, that is, which problem instances can guarantee that the abstraction is valid.

The abstraction-refinement framework is an effective approach for verifying the properties of multi-agent systems. The framework first builds an abstract model based on 3-value semantics to preliminarily judge the system properties. If the property is true or false in the abstract model, the truth or falsehood of the property in the specific model can be directly determined; if it cannot be determined, it means that the abstract model is too rough and cannot accurately reflect the satisfaction of the property. At this point, the framework refines the abstract model by searching for failed states until a clear conclusion can be drawn. In order to achieve this process, a variety of abstraction and refinement strategies have been proposed based on this framework [**?** **?** **?** **?**]. Nevertheless, the abstraction-refinement framework cannot guarantee a complete abstract model, because it is undecidable to find the failure state in these two logics. In addition, the above works all focus on the abstraction of the model rather than the theoretical abstraction.

In this paper, we concentrate on ICGs that is more complex than the planning domain. The former involves two players while the latter only focuses on the single-agent case. Hence it is necessary to consider not only how the current player chooses actions, but also how the other player responds when we synthesize winning strategies. Recently, **?**] study the translation from generalized planning under partial observability to two-player games with perfect information. In the case there are no additional fairness assumptions, the former corresponds to two-player games with imperfect information. By adapting the belief-state construction, **?** remove imperfect information from games, and hence obtain a two-player game with perfect information corresponding to the

given generalized planning problem. However, they do not provide an approach to synthesizing winning strategies for the games with perfect information corresponding to the given generalized planning problem.

9. Conclusions and future work

We have devised an effective approach to computing generalized winning strategies that works for possibly infinite many instances of ICGs. The key insight of our approach is to generate a candidate formula that is consistent with two generated sets of positive states and negative states. To this end, we have put forward two approach to solve this problem. One adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Then, by integrating the SMT-based verification procedure, we have developed a CEGIS approach to synthesizing the winning formula and strategy of ICGs. We have implemented our approach and experimental results show the effectiveness and scalability of our proposed approach.

Our approach have some limitations, and hence leading to several avenues for future work. Firstly, the ICGs considered in this paper are formalized in LIA. For other ICGs, their formalization involves real numbers, existential quantifiers, multiple operator, rounding operator, exclusive-disjunction operator and so on. We would also like to consider the games whose formalizations require more expressive language. Secondly, for the same solvable test cases, the strategy generated based on exact binary classification is larger than the one generated by the enumeration method, while the winning strategy can be represented in a more concise form. We would like to design the solution to strategy synthesis based on more efficient algorithms. Finally, we focus on ICGs. It is also interesting to apply our approach to other games, *e.g.*, partizan combinatorial games (*i.e.*, some moves are available to one player and not to the other).

Appendix A. Domains

Appendix A.1. Subtraction Games

Appendix A.1.1. Take Away [?]

Description:. There is a pile of v chips. A move consists of taking any number from 1 to m chips. When no chip can be taken, the game ends.

Test cases:. Each test case differs in the parameter m which is at most 30. For each play rule, the number of test cases of take away game are 30.

Formalization:.

- $\mathcal{X} : \{v\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\mathcal{A} : \{take(k)\}$;
- $\texttt{pre}(take(k) : v \geq k \wedge k > 0 \wedge k \leq m$;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$.

Appendix A.1.2. Subtraction [?]

Description:. The Subtraction game is a generalization of the Take-away game. Let S be a finite set of positive integers. In this game, each player can take k chips from the pile where $k \in S$.

Test cases:. Each test case differs in the set S. We consider the set S with size at most 4. For the set with size less than 4, the value of each element of S is ranged from 1 to 10. There are 385 test cases in subtraction game.

Formalization:.

- $\mathcal{X} : \{v\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v \geq 0 \wedge v < \min(S)$;
- $\mathcal{A} : \{take(k)\}$;
- $\texttt{pre}(take(k)) : (\bigvee_{s \in S} k = s) \wedge v \geq \min(S)$;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$;

Appendix A.1.3. S, D-Mark [?]

Description:. S, D-Mark Game is a variant of take-away game. Given two finite sets of integers S and D. Each player can either remove k chips where $k \in S$ and $k \leq v$, or remove $v - \frac{v}{d}$ chips where $d \in D$ and d divides v.

Test cases:. For set S, we consider that the element is $[1, 2, \cdots, t-1]$, the value of t ranges from 2 to 9. For set D, We consider the single element d, and value ranges from 2 to 9. A total of 64 cases are considered.

Formalization:.

- $\mathcal{X} : \{v\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 2\}$;
- $\texttt{pre}(take_1(k)) : (\bigvee_{s \in S} k = s) \wedge v \geq \min(S)$;
- $\texttt{eff}(take_1(k)) : \{\langle \top, v, v - k \rangle\}$;
- $\texttt{pre}(take_2(k)) : \bigvee_{d \in D} k \cdot d = v$;
- $\texttt{eff}(take_2(k)) : \{\langle \top, v, k \rangle\}$.

Appendix A.2. Nim Games

Appendix A.2.1. Nim [?]

Description:. There are n piles of chips where v_i denotes the number of chips of the i-th pile. Each player takes k chips from only one pile. If all piles are empty, then the game ends.

Test cases:. Because the winning formula of Nim game requires the exclusive-or operator, so it is not LIA-definable. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th piles of a legal state to be at most 5. That is, the maximum m of v_3 can be 0, 1, $\cdots$, or 5. We also considered the version with no restrictions on v_3. There are 7 test cases of this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 0$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.2. Monotonic Nim [?]

Description:. The Monotonic Nim is a variant of Nim game. The player can take arbitrary number of chips from one pile, given that the resulting sequence is non-decreasing ($v_i \geq v_{i+1}$). If all piles are empty, then the game ends.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge (\bigwedge_{i=1}^{2} v_i \geq v_{i+1}) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 0 \wedge v_i - k \geq v_{i+1}$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.3. Empty and Transfer [?]

Description:. Empty and Transfer is a variant of Nim game. A move consists of emptying one of the piles, selecting another pile and transferring some but not all of the chips from it into the empty pile.

Test cases:. We restrict the number of piles to be 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 1$;
- $\mathcal{A} : \{empty_i(k_1, k_2) \mid 1 \leq i \leq 3\}$;
- $\texttt{pre}(empty_1(k_1, k_2)) : ((k_1 = 2 \wedge v_2 \geq k_2) \vee (k_1 = 3 \wedge v_3 \geq k_2)) \wedge k_2 \geq 0$;
- $\texttt{eff}(empty_1(k_1, k_2)) : \{\langle \top, v_1, k_2 \rangle, \langle k_1 = 2, v_2, v_2 - k_2 \rangle, \langle k_1 = 3, v_3, v_3 - k_2 \rangle\}$;

- $\texttt{pre}(empty_2(k_1, k_2)) : ((k_1 = 1 \wedge v_1 \geq k_2) \vee$
  $(k_1 = 3 \wedge v_3 \geq k_2)) \wedge k_2 \geq 0$;
- $\texttt{eff}(empty_2(k_1, k_2)) : \{\langle \top, v_2, k_2 \rangle,$
  $\langle k_1 = 1, v_1, v_1 - k_2 \rangle, \langle k_1 = 3, v_3, v_3 - k_2 \rangle\}$;
- $\texttt{pre}(empty_3(k_1, k_2)) : ((k_1 = 2 \wedge v_2 \geq k_2) \vee$
  $(k_1 = 1 \wedge v_1 \geq k_2)) \wedge k_2 \geq 0$;
- $\texttt{eff}(empty_3(k_1, k_2)) : \{\langle \top, v_3, k_2 \rangle,$
  $\langle k_1 = 1, v_1, v_1 - k_2 \rangle, \langle k_1 = 2, v_2, v_2 - k_2 \rangle\}$.

Appendix A.2.4. Empty All But One [?]

Description:. Empty All But One is a variant of Nim game. A move of the game consists of emptying $n - 1$ of the piles and redistributing the chips of the remaining piles among all the piles in such a way that there is at least one chip in each pile. The game ends when the number of chips in each pile is less than n.

Test cases:. We restrict the number of piles to be 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 6 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i < 3$;
- $\mathcal{A} : \{keep_i(k_1, k_2, k_3) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(keep_i(k_1, k_2, k_3)) : (\bigwedge_{i=1}^{3} k_i > 0) \wedge$
  $(k_1 + k_2 + k_3) \leq v_i$;
- $\texttt{eff}(keep_i(k_1, k_2, k_3)) : \{\langle \top, v_1, k_1 \rangle,$
  $\langle \top, v_2, k_2 \rangle, \langle \top, v_3, k_3 \rangle\}$.

Appendix A.2.5. Empty and Redistribute [?]

Description:. Empty and Redistribute is a variant of the Nim game. A move in the game consists of two steps. Firstly, one of the piles is chosen and all the chips of this pile are discarded. Secondly, the remaining chips in the other $n-1$ piles are redistributed among the n piles in such a way that there is at least one chip in each pile. The game ends when the the total number of chips is less than n.

Test cases:. We restrict the number of piles to be 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 6 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i > 0) \wedge v_3 \leq m$;
- $\mathcal{E} : v_1 + v_2 + v_3 \leq 3$;
- $\mathcal{A} : \{empty_1(k_1, k_2, k_3)\}$;
- $\texttt{pre}(empty_1(k_1, k_2, k_3)) : (\bigwedge_{i=1}^{3} k_i > 0) \wedge v_1 + v_2 + v_3 - v_i \geq 3 \wedge k_1 + k_2 + k_3 = v_1 + v_2 + v_3 - v_i$;
- $\texttt{eff}(empty_1(k_1, k_2, k_3)) : \{\langle \top, v_1, k_1 \rangle, \langle \top, v_2, k_2 \rangle, \langle \top, v_3, k_3 \rangle\}$.

Appendix A.2.6. Entropy Reduction Game [?]

Description:. Empty Reduction is a variant of Nim game. A move consists of transferring chips from one pile to another in such a way as to reduce the entropy. That is, a move is to take chips from one pile and put them in another provided the piles are made more nearly equal in size.

Test cases:. We restrict the number of piles to be 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i > 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} \bigwedge_{j=1, i \neq j}^{3} v_i - v_j \leq 1$;
- $\mathcal{A} : \{take_i(k_1, k_2) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_1(k_1, k_2)) : (k_1 = 2 \wedge v_1 > v_2 \wedge k_2 > 0 \wedge k_2 < v_1 - v_2) \vee (k_1 = 3 \wedge v_1 > v_3 \wedge k_2 > 0 \wedge k_2 < v_1 - v_3)$;
- $\texttt{eff}(take_1(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_2 \rangle, \langle k_1 = 2, v_2, v_2 + k_2 \rangle, \langle k_1 = 3, v_3, v_3 + k_2 \rangle\}$;
- $\texttt{pre}(take_2(k_1, k_2)) : (k_1 = 1 \wedge v_2 > v_1 \wedge k_2 > 0 \wedge k_2 < v_2 - v_1) \vee (k_1 = 3 \wedge v_2 > v_3 \wedge k_2 > 0 \wedge k_2 < v_2 - v_3)$;
- $\texttt{eff}(take_2(k_1, k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle, \langle k_1 = 1, v_1, v_1 + k_2 \rangle, \langle k_1 = 3, v_3, v_3 + k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : (k_1 = 1 \wedge v_3 > v_1 \wedge k_2 > 0 \wedge k_2 < v_3 - v_1) \vee (k_1 = 2 \wedge v_3 > v_2 \wedge k_2 > 0 \wedge k_2 < v_3 - v_2)$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_3, v_3 - k_2 \rangle, \langle k_1 = 1, v_1, v_1 + k_2 \rangle, \langle k_1 = 2, v_2, v_2 + k_2 \rangle\}$.

Appendix A.2.7. Min Game [?]

Description:. There are two pairs of chips v_1 and v_2. A move consists of removing k chips from the larger pile where k no greater than the smaller pile. The game ends when one of the piles reduce to zero.

Test cases:. We restrict the maximum number m of v_2 to be at most 5. Plus the unrestricted version of v_2, there are 6 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \vee v_2 = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_1(k)) : v_1 \geq v_2 \wedge k \leq v_2 \wedge k > 0$;
- $\texttt{eff}(take_1(k)) : \{\langle \top, v_1, v_1 - k \rangle\}$;
- $\texttt{pre}(take_2(k)) : v_1 \leq v_2 \wedge k \leq v_1 \wedge k > 0$;
- $\texttt{eff}(take_2(k)) : \{\langle \top, v_2, v_2 - k \rangle\}$.

Appendix A.2.8. l-Slow Nim [?]

Description:. The l-Slow Nim is a generalization of the Nim game where l is a positive number. The number of chips to be remove is to restricted in the interval $[1, l]$.

Test cases:. We restrict the number of piles to be 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. We also restrict the maximum m_1 of l to be 30. There are 180 test cases of this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k \leq m_1$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.9. Sharing Nim [?]

Description:. There are n piles of chips where v_i denotes the number of chips of the i-th pile. In one turn, the player takes some chips from one pile and adds them to the other pile provided that after the move the destination pile does not have more chips than the source pile. The game ends when there are no actions to perform.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th piles to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} \bigwedge_{j=1, i \neq j}^{3} v_i - v_j \leq 1$;
- $\mathcal{A} : \{take_1(k_1, k_2), take_2(k_1, k_2), take_3(k_1, k_2)\}$;
- $\texttt{pre}(take_1(k_1, k_2)) : ((k_1 = 2 \wedge v_1 - k_2 \geq v_2 + k_2) \vee$
  $(k_1 = 3 \wedge v_1 - k_2 \geq v_3 + k_2)) \wedge k_2 > 0$;
- $\texttt{eff}(take_1(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_2 \rangle,$
  $\langle k_1 = 2, v_2, v_2 + k_2 \rangle, \langle k_1 = 3, v_3, v_3 + k_2 \rangle\}$;
- $\texttt{pre}(take_2(k_1, k_2)) : ((k_1 = 1 \wedge v_2 - k_2 \geq v_1 + k_2) \vee$
  $(k_1 = 3 \wedge v_2 - k_2 \geq v_3 + k_2)) \wedge k_2 > 0$;
- $\texttt{eff}(take_2(k_1, k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle,$
  $\langle k_1 = 1, v_1, v_1 + k_2 \rangle, \langle k_1 = 3, v_3, v_3 + k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : ((k_1 = 1 \wedge v_3 - k_2 \geq v_1 + k_2) \vee$
  $(k_1 = 2 \wedge v_3 - k_2 \geq v_2 + k_2)) \wedge k_2 > 0$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_3, v_3 - k_2 \rangle,$
  $\langle k_1 = 1, v_1, v_1 + k_2 \rangle, \langle k_1 = 2, v_2, v_2 + k_2 \rangle\}$.

Appendix A.2.10. Small Nim [?]

Description:. Small Nim is a restriction of Nim game. Each player only remove chips from the pile with the smallest (or any equal smallest) chips.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th piles to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0 \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : (\bigwedge_{j=1, i \neq j}^{3} v_i \leq v_j) \wedge v_i \geq k \wedge k > 0$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.11. Greedy Nim [?]

Description:. Greedy Nim is a dual of Small Nim game. Each player only remove chips from the pile with the largest (or any equal largest) chips.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th piles to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0 \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;

- $\texttt{pre}(take_i(k)) : (\bigwedge_{j=1,j\neq i}^{3} v_i \geq v_j) \wedge v_i \geq k \wedge k > 0$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k\rangle\}$.

Appendix A.2.12. Odd-or-Even Nim [?]

Description:. Odd-or-Even Nim is a restriction of the Nim game. In the beginning, the first player has to specify whether the opponent is to take an odd or an even number of chips. In the following rounds, each player first takes arbitrary number of chips from the chosen pile that complies with the current restriction, and then specifies the restriction on the next round.

Test cases:. We restrict the number of piles to be at most 2 and the maximum number m of chips on the 2-th piles to be at most 5. Plus the unrestricted version of v_2, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{r, v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge r \geq 0 \wedge r \leq 2 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0 \wedge r \geq 0 \wedge r \leq 2$;
- $\mathcal{A} : \{specify(k), take_e(k), take_o(k)\}$;
- $\texttt{pre}(specify(k)) : r = 2 \wedge (k = 0 \vee k = 1)$;
- $\texttt{eff}(specify(k)) : \{\langle \top, r, k\rangle\}$;
- $\texttt{pre}(take_{1e}(k)) : r \neq 2 \wedge k > 0 \wedge k \equiv_2 r \wedge v_1 \geq k$;
- $\texttt{eff}(take_{1e}(k)) : \{\langle \top, r, 0\rangle, \langle \top, v_1, v_1 - k\rangle\}$;
- $\texttt{pre}(take_{1o}(k)) : r \neq 2 \wedge k > 0 \wedge k \equiv_2 r \wedge v_1 \geq k$;
- $\texttt{eff}(take_{1o}(k)) : \{\langle \top, r, 1\rangle, \langle \top, v_1, v_1 - k\rangle\}$;
- $\texttt{pre}(take_{2e}(k)) : r \neq 2 \wedge k > 0 \wedge k \equiv_2 r \wedge v_2 \geq k$;

- $\texttt{eff}(take_{2e}(k)) : \{\langle \top, r, 0\rangle, \langle \top, v_2, v_2 - k\rangle\}$;
- $\texttt{pre}(take_{2o}(k)) : r \neq 2 \wedge k > 0 \wedge k \equiv_2 r \wedge v_2 \geq k$;
- $\texttt{eff}(take_{2o}(k)) : \{\langle \top, r, 1\rangle, \langle \top, v_2, v_2 - k\rangle\}$.

Appendix A.2.13. Modular Nim [?]

Description:. Modular Nim is a generalization of the Nim game. Each player can remove k chips from one pile where $k \not\equiv_n x, 0 \leq x < n$.

Test cases:. We restrict the number of piles n to be at most 3 and $0 \leq x < n$. There are 5 test cases.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 0 \wedge k \not\equiv_n x$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k\rangle\}$.

Appendix A.2.14. Pointed Nim [?]

Description:. Pointed Nim is an extension of Nim game augmented with a pointer. This pointer designates which pile in the next move to be take. To make a move a player removes one or more chips from the designated heap, and then must move the pointer to a different, non-empty heap. Initially, the pointer is set to 0 meaning that the first player can take chips from any one pile.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th pile to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{r, v_1, v_2, v_3\}$;
- $\mathcal{C} : (\bigwedge_{i=1}^{3} v_i \geq 0) \wedge r \geq 0 \wedge r \leq 3 \wedge v_3 \leq m$;
- $\mathcal{E} : (\bigwedge_{i=1}^{3} v_i = 0) \wedge r > 0 \wedge r \leq 3$;
- $\mathcal{A} : \{take_i(k_1, k_2) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k_1, k_2)) : v_i \geq k_1 \wedge k_1 > 0 \wedge$
  $(r = i \vee r = 0)$;
- $\texttt{eff}(take_i(k_1, k_2)) : \{\langle r = 0 \vee r = 1, v_1, v_1 - k_1 \rangle,$
  $\langle r = 0 \vee r = 2, v_2, v_2 - k_1 \rangle,$
  $\langle i = 0 \vee i = 3, v_3, v_3 - k_1 \rangle, \langle \top, r, k_2 \rangle\}$.

Appendix A.2.15. End Nim [?]

Description:. End Nim is a variant of Nim game. There are n piles of chips in order. Each player can only take chips from the front end or back end pile.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0 \wedge v_3 \leq m$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 1 \wedge ((\bigwedge_{j=1}^{i-1} v_j = 0) \vee$
  $(\bigwedge_{j=i+1}^{3} v_j = 0))$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.16. Loop End Nim [?]

Description:. Loop (Leave only one pile) End Nim is a bit different from End Nim except the ending condition. If the game reduces to a single pile, then the game ends.

Test cases:. We restrict the number of piles to be at most 3 and the maximum number m of chips on the 3-th pile of a legal state to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0 \wedge v_3 \leq m$;
- $\mathcal{E} : (v_1 = 0 \wedge v_2 > 0 \wedge v_3 = 0) \vee (v_1 > 0 \wedge v_2 = 0 \wedge v_3 = 0) \vee (v_1 = 0 \wedge v_2 = 0 \wedge v_3 > 0)$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 0 \wedge ((\bigwedge_{j=1}^{i-1} v_j = 0) \vee (\bigwedge_{j=i+1}^{3} v_j = 0))$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle\}$.

Appendix A.2.17. Staircase Nim [?]

Description:. A staircase of n steps contains coins on some of the steps where v_i denotes the number of coins of the i-th steps. A move of Staircase Nim consists of moving any positive number of coins from any step v_i, to the next lower step v_{i-1}. Coins reaching the ground (step 0) are removed from play. The game ends when all coins are on the ground.

Test cases:. We restrict the number of steps to be at most 3 and the maximum number m of coins on the 3-th step to be at most 5. Plus the unrestricted version of v_3, there are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{3} v_i \geq 0$;
- $\mathcal{E} : \bigwedge_{i=1}^{3} v_i = 0$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(take_1(k)) : v_1 \geq k \wedge k > 1$;
- $\texttt{eff}(take_1(k)) : \{\langle \top, v_1, v_1 - k \rangle\}$;
- $\texttt{pre}(take_i(k)) : v_i \geq k \wedge k > 1$;
- $\texttt{eff}(take_i(k)) : \{\langle \top, v_i, v_i - k \rangle, \langle \top, v_{i-1}, v_{i-1} + k \rangle\}$

Appendix A.2.18. Circular Nim [?]

Description:. There n piles of chips placed in a circle. A move consists of choosing m consecutive piles where $m \leq n$, and taking at least one chip from one or more of the m stacks. If all piles are empty, then the game ends.

Test cases:. We consider the test cases where n ranges from 1 to 5 and $0 \leq m \leq n$. There are 15 test cases of this game.

Formalization:.

- $\mathcal{X} : \{v_1, \cdots, v_n\}$;
- $\mathcal{C} : \bigwedge_{i=1}^{n} v_i \geq 0$;
- $\mathcal{E} : \bigwedge_{i=1}^{n} v_i = 0$;
- $\mathcal{A} : \{take_i(k_1, \cdots, k_m) | 1 \leq i \leq n\}$;
- $\texttt{pre}(take_i(k_1, \cdots, k_m)) : (\bigwedge_{i=1}^{m} k_m \geq 0) \wedge$
  $\sum_{i=1}^{m} k_i > 1 \wedge (\bigwedge_{j=1}^{m} v_{(i+j-1)\%n} \geq k_j)$;
- $\texttt{eff}(take_i(k_1, \cdots, k_m)) : \{\langle \top, v_i, v_i - k_1 \rangle, \cdots,$
  $\langle \top, v_{(i+m-1)\%n}, v_{(i+m-1)\%n} - k_m \rangle\}$.

Appendix A.3. Welter Games

Appendix A.3.1. Welter [?]

Description:. There are n coins placed on distinct squares of a semi-infinite linear board and the squares numbered $0, 1, 2, \cdots$. The variable v_i denotes the position of the i-th coin. Two players alternately moves a coin to a lower unoccupied square. The game ends when the player is unable to move.

Test cases:. We restrict the number of coins to be 2. There is 1 test case in this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_1 \neq v_2$;
- $\mathcal{E} : v_1 + v_2 = 1$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 2\}$;
- $\texttt{pre}(take_1(k)) : v_1 \geq k \wedge k > 0 \wedge (v_1 - k) \neq v_2$;
- $\texttt{eff}(take_1(k)) : \{\langle \top, v_1, v_1 - k \rangle\}$;
- $\texttt{pre}(take_2(k)) : v_2 \geq k \wedge k > 0 \wedge (v_2 - k) \neq v_1$;
- $\texttt{eff}(take_2(k)) : \{\langle \top, v_2, v_2 - k \rangle\}$.

Appendix A.3.2. l-welter [?]

Description:. l-welter Game is a generalization of Welter's Game. Each move player could move a coin at most l squares from its present position.

Test cases:. We restrict the number of piles to be 2 and the maximum m_1 of l to be 5. There are 5 test cases of this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_1 \neq v_2$;
- $\mathcal{E} : v_1 + v_2 = 1$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 2\}$;
- $\texttt{pre}(take_1(k)) : v_1 \geq k \wedge k > 0 \wedge k < m_1 \wedge$
  $(v_1 - k) \neq v_2$;
- $\texttt{eff}(take_1(k)) : \{\langle \top, v_1, v_1 - k \rangle\}$;
- $\texttt{pre}(take_2(k)) : v_2 \geq k \wedge k > 0 \wedge k < m_1 \wedge$
  $(v_2 - k) \neq v_1$;
- $\texttt{eff}(take_2(k)) : \{\langle \top, v_2, v_2 - k \rangle\}$.

*Appendix A.3.3. Max-Welter [***?** *]*

Description:. Max-Welter is a restriction of the Welter game. Two players alternately moves the coin in the largest square to a lower unoccupied square.

Test cases:. We restrict the number of coins to be 2. There is 1 test case in this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_1 \neq v_2$;
- $\mathcal{E} : v_1 + v_2 = 1$;
- $\mathcal{A} : \{take_i(k) | 1 \leq i \leq 2\}$;
- $\texttt{pre}(take_1(k)) : v_1 \geq v_2 \wedge v_1 \geq k \wedge k > 0 \wedge$
  $(v_1 - k) \neq v_2$;

- $\texttt{eff}(take_1(k)) : \{\langle \top, v_1, v_1 - k \rangle\}$;
- $\texttt{pre}(take_2(k)) : v_2 \geq v_1 \wedge v_2 \geq k \wedge k > 0 \wedge$
  $(v_2 - k) \neq v_1$;
- $\texttt{eff}(take_2(k)) : \{\langle \top, v_2, v_2 - k \rangle\}$.

Appendix A.4. Wythoff Games

Appendix A.4.1. Wythoff [?]

Description:. There are exactly two piles of chips v_1 and v_2. Each player can take k_1 chips from the first pile or k_2 chips from the second pile as Nim game (namely, Nim move). He also can take chips from both piles (namely, Wythoff move). In Wythoff move, the player must take the same number of chips from both piles. If both piles are empty, then the game ends.

Test cases:. The formalization are as follows, and other wythoff games are similar. We restrict the maximum m of v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 7 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : k_1 > 0 \wedge k_1 \leq v_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : k_2 > 0 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;

- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge v_2 \geq k_2 \wedge$
  $k_1 > 0 \wedge k_1 = k_2$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.2. l-Wythoff [?]

Description:. l-Wythoff Game is a variant of Wythoff game where l is a non-negative integer. In a Wythoff move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $|k_1 - k_2| < l$.

Test cases:. We restrict the maximum m_1 of l to be at most 5 and the maximum m of v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 35 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : k_1 > 0 \wedge k_1 \leq v_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : k_2 > 0 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge k_1 > 0 \wedge v_2 \geq k_2 \wedge$
  $k_2 > 0 \wedge ((k_1 > k_2 \wedge k_1 - k_2 < m_1) \vee$
  $(k_2 > k_1 \wedge k_2 - k_1 < m_1))$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.3. Monotonic l-Diet Wythoff [?]

Description:. This game is a monotonic variant of Wythoff game requiring $v_1 \geq v_2$ and the number of chips to remove cannot exceed l.

Test cases:. We restrict the maximum m_1 of l to be at most 5 and the maximum m of v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 35 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_2 \geq 0 \wedge v_1 \geq v_2 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0, v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : k_1 > 0 \wedge k_1 \leq m_1 \wedge v_1 \geq v_2 + k_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : k_2 > 0 \wedge k_2 \leq m_1 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : v_2 \geq k_2 \wedge v_1 \geq k_1 \wedge$
  $k_1 > 0 \wedge k_1 < m_1 \wedge k_1 = k_2$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.4. Wythoff with subtraction set S [?]

Description:. Let S be a finite set of positive integers. Each player take $k \in S$ chips from each of piles in a Wythoff move.

Test cases:. We consider the set S with size at most 2 and the maximum m of v_2 to be at most 5. The value of each element of S ranges from 1 to 5. We also considered the version with no restrictions on v_2. There are 105 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : v_1 \geq k_1 \wedge k_1 > 0$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : v_2 \geq k_2 \wedge k_2 > 0$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : (\bigvee_{s \in S} k_1 = s) \wedge v_1 \geq k_1 \wedge$
  $v_2 \geq k_2 \wedge k_1 = k_2$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.5. Wythoff with a function f [?]

Description:. This game is a variant of Wythoff game. In the Wythoff move, player could take k_1 chips from the first pile and k_2 chips from the second one where k_1 and k_2 is restricted by the condition: $0 < k_1 \leq k_2 < f(k_1)$.

Test cases:. We restrict the maximum m of v_2 to be at most 5 and the function $f(k) = k+i$ where $0 \leq i \leq 5$. We also considered the version with no restrictions on v_2. There are 42 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : v_1 \geq 0 \wedge k_1 > 0 \wedge k_1 \leq v_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : v_2 \geq 0 \wedge k_2 > 0 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge v_2 \geq k_2 \wedge$
  $k_1 > 0 \wedge k_2 > 0 \wedge k_2 < f(k) \wedge k_1 \leq k_2$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.6. 1st (l_1, l_2)-restricted Wythoff [?]

Description:. This is the first restriction of Wythoff game with two nonegative integers l_1 and l_2. Here, $l_1 \leq l_2$. After a Wythoff move, the new state (v_1, v_2) must satisfies $\min(v_1, v_2) \geq l_1$ and $\max(v_1, v_2) \geq l_2$.

Test cases:. We restrict the maximum m_1 and m_2 of the integers l_1, l_2 to be 5 and the maximum number m of piles v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 105 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;

- $\mathcal{E} : v_1 = 0, v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : k_1 > 0 \wedge k_1 \leq v_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : k_2 > 0 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge v_2 \geq k_2 \wedge k_1 > 0 \wedge$
  $k_1 = k_2 \wedge (v_1 - k_1 \geq m_1 \vee v_2 - k_2 \geq m_1) \wedge$
  $(v_1 - k_1 \geq m_2 \vee v_2 - k_2 \geq m_2)$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.7. 2nd (l_1, l_2)-restricted Wythoff [?]

Description:. This is the second restriction of Wythoff game with two nonegative integers l_1 and l_2. In one move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $\min(k_1, k_2) < l_1$ or $|k_1 - k_2| < l_2$.

Test cases:. We restrict the maximum m_1 and m_2 of the integers l_1, l_2 to be 5 and the maximum number m of piles v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 150 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : \{take(k_1, k_2)\}$;

- $\texttt{pre}(take(k_1, k_2)) : v_1 \geq k_1 \wedge k_1 \geq 0 \wedge v_2 \geq k_2 \wedge$
  $k_2 \geq 0 \wedge k_1 + k_2 > 0 \wedge (k_1 < m_1 \vee$
  $k_2 < m_1 \vee (k_1 \geq k_2 \wedge k_1 - k_2 < m_2) \vee$
  $(k_2 \geq k_1 \wedge k_2 - k_1 < m_2)$;
- $\texttt{eff}(take(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.8. 3rd (l_1, l_2)-restricted Wythoff [?]

Description:. This is the third restriction of Wythoff game with two positive integers l_1 and l_2. In a Wythoff move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $0 \leq |k_2 - k_1| < (l_1 - 1)\lambda + l_2, \lambda = \min(k_1, k_2)$.

Test cases:. We restrict the maximum m_1 and m_2 of the integers l_1, l_2 to be 5 and the maximum number m of piles v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 175 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0 \wedge v_2 \leq m$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : v_1 \geq k_1 \wedge k_1 > 0$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : v_2 \geq k_2 \wedge k_2 > 0$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;

- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge k_1 > 0 \wedge v_2 \geq k_2 \wedge$
  $k_2 > 0 \wedge ((k_2 \geq k_1 \wedge k_2 - k_1 < (m_1 - 1)k_1 +$
  $m_2) \vee (k_1 \geq k_2 \wedge k_1 - k_2 < (m_1 - 1)k_2 + m_2))$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.4.9. 4th (l_1, l_2)-restricted Wythoff [?]

Description:. This is the fourth restriction of Wythoff game with two positive integers l_1 and l_2. In a Wythoff move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $k_1 \cdot l = k_2 \cdot m$.

Test cases:. We restrict the maximum m_1 and m_2 of the integers l_1, l_2 to be 5 and the maximum number m of piles v_2 to be at most 5. We also considered the version with no restrictions on v_2. There are 140 test cases in total.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 \geq 0 \wedge v_2 \geq 0$;
- $\mathcal{E} : v_1 = 0 \wedge v_2 = 0 \wedge v_2 \leq m$;
- $\mathcal{A} : take_1(k_1), take_2(k_2), take_3(k_1, k_2)$;
- $\texttt{pre}(take_1(k_1)) : k_1 > 0 \wedge k_1 \leq v_1$;
- $\texttt{eff}(take_1(k_1)) : \{\langle \top, v_1, v_1 - k_1 \rangle\}$;
- $\texttt{pre}(take_2(k_2)) : k_2 > 0 \wedge k_2 \leq v_2$;
- $\texttt{eff}(take_2(k_2)) : \{\langle \top, v_2, v_2 - k_2 \rangle\}$;
- $\texttt{pre}(take_3(k_1, k_2)) : v_1 \geq k_1 \wedge k_1 > 0 \wedge v_2 \geq k_2 \wedge$
  $k_2 > 0 \wedge k_1 \cdot m_1 = k_2 \cdot m_2$;
- $\texttt{eff}(take_3(k_1, k_2)) : \{\langle \top, v_1, v_1 - k_1 \rangle,$
  $\langle \top, v_2, v_2 - k_2 \rangle\}$.

Appendix A.5. Chomp Games

Appendix A.5.1. Chomp [?]

Description:. Cookies are laid out on n rows where v_i denotes the number of cookies of the i-th row. The number of cookies is non-decreasing, that is, if $i > j$, then $v_i \leq v_j$. The cookie in the top-left position $(1, 1)$ is poisoned. The loser is the player who has no choice but to eat the poisoned cookie. At each move, a player has to eat a remaining cookie, together with all cookies to the right and above it.

Test cases:. To the best of our knowledge, the winning strategy for Chomp game is unknown. So we restrict the number of rows to be at most 3 and the maximum number m of cookies on the 3-th row to be at most 5. That is, the maximum of v_3 is ranged from 0 to 5. A total of 7 test cases with different restrictions on the maximum of v_3 are considered.

Formalization:.

- $\mathcal{X} : \{v_1, v_2, v_3\}$;
- $\mathcal{C} : v_1 > 0 \wedge v_2 \geq 0 \wedge v_3 \geq 0 \wedge v_3 \leq m$;
- $\mathcal{E} : v_1 = 1 \wedge v_2 = 0 \wedge v_3 = 0$;
- $\mathcal{A} : \{eat_i(k) | 1 \leq i \leq 3\}$;
- $\texttt{pre}(eat_1(k)) : v_1 \geq k \wedge k > 1$;
- $\texttt{eff}(eat_1(k)) : \{\langle \top, v_1, k-1\rangle, \langle v_2 \geq k, v_2, k-1\rangle,$
  $\langle v_3 \geq k, v_3, k-1\rangle\}$;
- $\texttt{pre}(eat_2(k)) : v_2 \geq k \wedge k > 0$;
- $\texttt{eff}(eat_2(k)) : \{\langle \top, v_2, k-1\rangle, \langle v_3 \geq k, v_3, k-1\rangle\}$;
- $\texttt{pre}(eat_3(k)) : v_3 \geq k \wedge k > 0$;
- $\texttt{eff}(eat_3(k)) : \{\langle \top, v_3, k-1\rangle\}$.

Appendix A.5.2. L-shaped Chomp [?]

Description:. The L-shaped Chomp game is the Chomp game with an L-shaped board with vertical and horizontal lines. The state variables v_1 and v_2 denote numbers of chips on the vertical and horizontal lines respectively.

Test cases:. There is only one test case of this game.

Formalization:.

- $\mathcal{X} : \{v_1, v_2\}$;
- $\mathcal{C} : v_1 > 0 \wedge v_2 > 0$;
- $\mathcal{E} : v_1 = 1 \wedge v_2 = 1$;
- $\mathcal{A} : \{eat_i(k) | 1 \leq i \leq 2\}$;
- $\texttt{pre}(eat_1(k)) : v_1 \geq k \wedge k > 1$;
- $\texttt{eff}(eat_1(k)) : \{\langle \top, v_1, k-1 \rangle\}$;
- $\texttt{pre}(eat_2(k)) : v_2 \geq k \wedge k > 1$;
- $\texttt{eff}(eat_2(k)) : \{\langle \top, v_2, k-1 \rangle\}$.